Speed Synchronization of a Dual-BLDC Tracked Differential-Drive Vehicle Using a GA-Tuned Multi-Rate Cross-Coupled Controller

Md Monirul Islam[a,*], Mainul Islam[b] and Md Shakil Tanvir[c]

[a]Department of Electrical and Electronic Engineering, Bangladesh University of Engineering and Technology, Dhaka, Bangladesh
[b]Department of Electrical and Electronic Engineering, Islamic University of Technology, Gazipur, Bangladesh
[c]Department of Electrical and Electronic Engineering, Ahsanullah University of Science and Technology, Dhaka, Bangladesh

ARTICLE INFO

Keywords:
Speed synchronization
Cross-coupled control
Brushless DC motor
Genetic-algorithm tuning
Skid-steer vehicle
Embedded control
Multi-rate control

ABSTRACT

This paper studies distributed speed synchronization for a tracked skid-steer vehicle driven by two three-phase BLDC "auto-rickshaw" motors, evaluated *on the assembled* 120 kg *vehicle*. On such a vehicle any uncommanded difference between the two track speeds degrades path-holding and integrates into heading drift, driven by the drives' *electrical* mismatch between low-cost brushless DC (BLDC) motors and, often more strongly, by *mechanical* asymmetries and one-sided loads. Each motor runs a local proportional–integral–derivative (PID) loop on its own microcontroller; a bilateral cross-coupled term, exchanged over a dedicated 1 ms inter-microcontroller link and updated at 1 kHz, drives the inter-motor difference toward zero, with speed feedback a tracking-observer estimate from a capacitive absolute encoder. Gains are tuned by a genetic algorithm in two stages—the per-motor loops off-ground, then the synchronization loop against the operating differential—because the inter-drive mechanical mismatch cannot be modeled beforehand. Evaluated over three tuning seeds on *held-out* operating points and a one-sided load, synchronization cuts the inter-motor error by 41 % to 65 % versus an independent-drive baseline, the clean-tracking gain exceeding twice the pooled run-to-run SD on a single test course. A same-plant comparison isolates the central finding: the 1 kHz *multi-rate* loop—made usable by the low-latency observer—is the performance lever, and its useful rate is bounded by the estimator bandwidth; a conventional cross-coupled law made multi-rate matches our controller, and the transparent PID is competitive with a super-twisting sliding-mode baseline, on commodity microcontrollers.

1. Introduction

Differential-drive and skid-steer ground vehicles steer by controlling the relative speed of their left and right drives, so keeping the two track speeds matched is a control objective in its own right: any uncommanded speed difference both degrades path-holding and integrates over distance into heading drift. On a low-cost platform the two drives *differ* enough that this difference is driven continuously by disturbances—drive mismatch and one-sided loads—which tuning each drive's own speed loop more tightly cannot remove, because neither loop sees the other. For an unmanned ground vehicle (UGV) expected to hold heading without continuous operator correction, the two loops must instead be coordinated by an inter-drive controller that rejects the differential in real time.

The problem is aggravated when the two drives *differ*, as on a tracked vehicle they do in two ways. *Electrically*, low-cost brushless DC (BLDC) motors — such as the auto-rickshaw motors used here — vary unit-to-unit in winding resistance and back-EMF constant, so at equal command the two drives accelerate and cruise differently: a persistent, structured bias over the entire run. *Mechanically*, the two sides differ in track-belt tension, ground contact, and wear, and one track may meet terrain resistance the other does not; such one-sided loads are frequently a *larger* differential disturbance than the electrical mismatch. Neither is visible to a single drive's own speed loop — both bias one track relative to the other — so rejecting them in real time is the role of an inter-drive *speed-synchronization* controller feeding the speed difference back into both drives (Koren, 1980; Feng, Koren and Borenstein, 1993).

Cross-coupled speed synchronization is well established, but the existing evidence does not transfer cleanly to this setting. Most multi-motor synchronization studies are validated in simulation or on matched permanent-magnet synchronous machines (PMSMs) on a bench under a single controller; the more advanced robust schemes (sliding-mode, model-predictive, or consensus control) synchronize excellently but rely on accurate plant models, state observers, or reliable real-time inter-agent communication that are hard to justify on a low-cost, distributed embedded target (Niu, Sun, Huang, Hu, Liang and Fang, 2023; Miao, He, Li and Wang, 2023; Li, Chai and Hou, 2025). Metaheuristic proportional–integral–derivative (PID) tuning is mature but almost always applied to a *single* motor (Mahmood, Ali, Mnati, Sabbar and Van Den Bossche, 2025; Garimella et al., 2012). Few works address a real, BLDC differential-drive vehicle realized with one microcontroller (MCU) per motor exchanging speed over a dedicated inter-MCU link (Megalingam, Manoharan, Mohandas and Kamalasanan, 2025; Reda, Onsy, Haikal and Ghanbari, 2025; Le, Tran and Nhat, 2022). This paper targets that combination and evaluates it on the assembled vehicle.

*Corresponding author
monirulislam.acad@gmail.com (M.M. Islam)
ORCID(s): 0009-0007-6537-9516 (M.M. Islam)

This paper presents:

- a *distributed dual-MCU* realization of bilateral cross-coupled speed synchronization for two three-phase BLDC drives, the motor microcontrollers exchanging speed over a dedicated 1 ms inter-MCU link while a master broadcasts the speed command over a separate bus;
- the central finding that a 1 kHz *multi-rate* synchronization loop is the performance lever, and that its useful rate is *bounded by the speed-estimate bandwidth*—which a low-latency capacitive-encoder tracking observer supplies—quantified by a same-plant loop-transfer analysis;
- a *two-stage* genetic-algorithm tuning procedure suited to the assembled vehicle—the per-motor loops calibrated off-ground, then the coupling refined against the operating differential—because the inter-drive mechanical mismatch cannot be modeled a priori;
- a quantified, seed-averaged reduction in inter-motor speed error versus an independent-drive baseline, and a *same-plant* comparison against conventional single-rate cross-coupling and a super-twisting sliding-mode controller that isolates the multi-rate architecture as the source of the gain.

The paper is organized as follows: Section 2 reviews synchronization structures and metaheuristic PID tuning; Section 3 the hardware platform; Section 4 the controller design and online GA tuning; Section 5 the dual-MCU implementation; Section 6 the on-vehicle results; and Section 7 concludes.

2. Related Work

2.1. Multi-Motor Synchronization Structures

Coordinated multi-motor control is organized along two axes: the *control structure* routing inter-motor information and the *control algorithm* acting on it (Niu et al., 2023; Zhang, Hu, Wang and Wang, 2024). Structurally, schemes divide into *non-coupling* forms—parallel control, where each motor independently tracks a shared command, and master–slave control, where followers track the master's output—and *coupling* forms that feed a synchronization error back between drives (Huang, Tu, Jiang, Li, Li and Zhang, 2018; Jerković Štil, Varga, Benšić and Barukčić, 2020). Non-coupling forms are simple but fragile: without cross-feedback a disturbance on one motor cannot be corrected by the others, and master–slave control is asymmetric, propagating a master disturbance to every follower while never returning a follower's (Niu et al., 2023; Zhang et al., 2024).

Coupling control originates with Koren's cross-coupled biaxial controller, which computes a contour (synchronization) error from both axes and injects a weighted correction into *both* loops (Koren, 1980); Feng *et al.* extended this to a differential-drive mobile robot, coupling the two wheel loops through the vehicle's orientation error (Feng et al., 1993). Later variants generalize the rule to n motors—improved cross-coupling with a softened reference (Chen, Liang and Shi, 2018), relative coupling that decouples synchronization from tracking through separate gains (Shi, Liu, Geng and Xia, 2016), deviation coupling driven by the average and sub-average speeds (Mu, Qi, Sun and Han, 2024), and ring coupling in which each motor synchronizes only with its neighbor (Li, Sun, Zhang and Yang, 2015)—while electronic line-shafting couples drives through a software "virtual shaft" (Lin, Cai, Yang and Zhang, 2017; Huang, Tu, Jiang, Pan and Zhu, 2021), with a long industrial lineage in paper machines (Valenzuela and Lorenz, 2001). Across these formulations, however, the *two-motor* case reduces to the same symmetric bilateral law $\beta_i = K(\omega_i - \omega_j)$: relative, deviation, and ring coupling collapse to it when n=2, and the passive-decomposition "shape" coordinate for two motors is the inter-motor position difference $\theta_1 - \theta_2$, whose rate is $\omega_1 - \omega_2$ (Jung and Kim, 2023). Our controller therefore adopts the *bilateral cross-coupled* structure rather than a higher-order coupling topology that offers no benefit at n=2.

2.2. Beyond PID: Robust and Advanced Synchronization

A large body of work pushes synchronization beyond linear PID, dominated by sliding-mode control (SMC) built on higher-order or terminal sliding-mode theory (Levant, 1993): global fast terminal SMC with cross-coupling for dual-motor systems (Zhu, Tu, Jiang, Pan and Huang, 2020; Wu, Zhang, Qi and Shi, 2025), non-singular fast terminal SMC with deviation coupling (Lan, Wang, Deng, Zhang and Song, 2023), adaptive SMC with ring coupling (Li et al., 2015), and cross-coupled fractional-order SMC for linear motors (Kuang, Gao and Tomizuka, 2020). Other strands add disturbance-observer-based fuzzy backstepping over a multi-agent graph (Dai, Zhang, Xu, Chen and Yan, 2022), unified finite-control-set model predictive control (MPC) with a PI plus fractional-order SMC inner loop (Miao et al., 2023), fixed-time leader–following consensus (Li et al., 2025), approximation-free prescribed-performance control (Liu, Lin, Yu, Rodríguez-Andina and Gao, 2022), decoupled robust control built on mechanical-parameter identification (Zhang, Wang and Fang, 2022), artificial potential fields with sliding-mode variable structure (Zhang, Niu, He, Zhao, Wu and Zhang, 2016), and cross-coupled sliding-mode speed synchronization within a hierarchical dual-BLDC steer-by-wire controller (Zou and Zhao, 2020). These methods can be highly effective—a unified MPC/SMC scheme drives peak dual-PMSM synchronization error from over 100 rpm to below 1 rpm under unbalanced load (Miao et al., 2023), and an identification-based robust law reports an order-of-magnitude lower synchronization error than PID on a gantry (Zhang et al., 2022). The cost is model dependence and implementation burden: accurate plant models, observer or sliding-surface design, Lyapunov/finite-time gain conditions, or real-time inter-agent communication, which the consensus literature itself flags as fragile to delay and packet

loss (Li et al., 2025). A recent review benchmarking field-oriented control (FOC), MPC, and SMC under matched conditions finds that PID-based FOC attains the best *steady-state* tracking and synchronization accuracy, the modern methods winning mainly in transient adjustment time (Niu et al., 2023); the fuzzy alternatives surveyed in (Jerković Štil et al., 2020) likewise trade interpretability for an expert-designed rule base. We therefore retain a transparent PID-plus-feedforward law—deployable on a low-cost, distributed microcontroller without a plant model or observer—and obtain performance through on-vehicle optimization and a fast bilateral synchronization loop. Section 6 compares it, on the same plant and equally tuned, against conventional cross-coupling (Koren, 1980) and a super-twisting sliding-mode controller (Levant, 1993).

2.3. Genetic-Algorithm and Metaheuristic PID Tuning

The value of a transparent PID hinges on its gains, and metaheuristic tuning is well established. Genetic algorithms (GAs) have tuned PID for BLDC speed control (Ibrahim, Mahmood and Sultan, 2019; Dakheel, Abdullah and Shneen, 2023; Mahmood et al., 2025) and position control (Garimella et al., 2012), for DC drives (Deraz, 2014), and—most recently—in combination with a sliding-mode observer for sensorless PMSM speed control (Abed, Hashim, Nasar, Hanfesh and Altahir, 2025); a broad review frames the field and its open problems (Joseph, Dada, Abidemi, Oyewola and Khammas, 2022), metaheuristics for BLDC speed control are benchmarked statistically in (Shafique, Fakhar, Rasool, Kashif and Suhail, 2024), and methodological studies guide GA crossover and mutation rates (Hassanat, Almohammadi, Alkafaween, Abunawas, Hammouri and Prasath, 2019). The reported gains are large: GA tuning reduces BLDC step overshoot from 54% to below 1% versus Ziegler–Nichols (Garimella et al., 2012), while a spec-driven weighted-sum objective outperforms generic IAE/MSE criteria (Deraz, 2014). Two limitations recur. First, almost all of this work tunes a *single* motor in *simulation*; applied to a wheeled robot, a metaheuristic optimizes one trajectory-tracking loop rather than inter-motor synchronization (Ha, Thuong and Truc, 2024), and the few multi-motor cases remain simulation-only (Wang, Du, Cui, Liu and Ma, 2023). Second, the optimization target is typically a single PID. We extend GA tuning to a *coupled two-motor* problem—jointly evolving each motor's PID, the synchronization controller, and the feedforward terms against a multi-scenario fitness—and tune them on the vehicle itself.

2.4. Differential-Drive Coordination and Embedded Realization

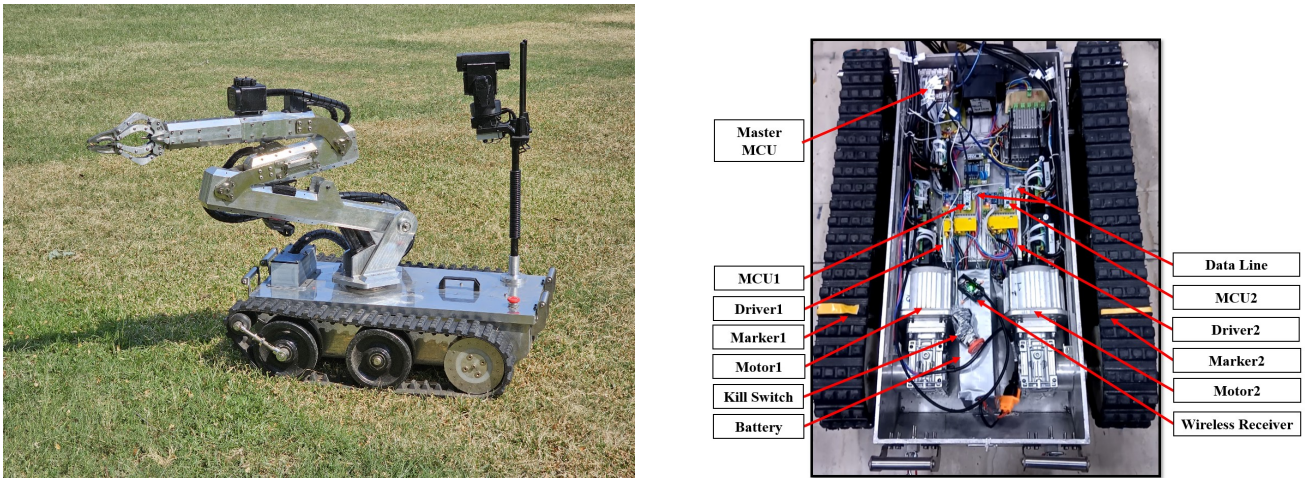
For wheeled and tracked vehicles, coordinating the dominant two-side independent drive is a core control problem (Jerković Štil et al., 2020; Feng et al., 1993). Reported approaches span PI wheel-speed synchronization on a differential-drive robot (Ashraf, Ahsan and Arshad, 2021), hybrid-metaheuristic PID with a multi-motor synchronization procedure on a four-wheel drive (Reda et al., 2025), a synchronous-error compensator for wheeled cross-coupled axles (Choi, Lee and Jung, 2024), master–slave versus cross-coupling on a two-motor bench (Le et al., 2022), and dedicated dual-BLDC controller hardware for a differential-drive rover (Megalingam et al., 2025). On the implementation side, low-cost microcontroller BLDC control with sensorless back-EMF commutation (Karnavas, Topalidis and Drakaki, 2019), STM32-based cascaded speed/current control (Zhao and Hua, 2022), and single-chip cross-coupling multi-motor drives (Amornwongpeeti, Ekpanyapong, Chayopitak, Monteiro, Martins and Afonso, 2015) demonstrate on-device feasibility. Running feedback loops at more than one sampling rate is itself a developed branch of sampled-data control—from the state-space and Nyquist theory of multivariable multirate systems (Araki and Yamamoto, 1986) to high-performance motion control, where the speed estimate and the actuator command deliberately run at different rates (Fujimoto, Hori and Kawamura, 2001; Fujimoto and Hori, 2002). We apply this idea to the *synchronization difference channel*: the cross-coupled loop runs faster than the per-motor loops, and Section 6 shows its useful rate is bounded not by the actuator but by the speed-estimate (observer) bandwidth. Our platform goes further architecturally: control is *distributed*, one microcontroller per motor, coupled by a dedicated inter-MCU universal asynchronous receiver/transmitter (UART) for synchronization and sharing a custom RS-485 master–slave (Modbus-style) command bus whose timing and framing we characterize in prior work (Tanvir, Ahmed, Islam, Islam, Hasan and Rashid, 2024). Each cited effort supplies one ingredient—a platform, a tuning method, a communication bus, or an embedded loop—but not their combination, and vehicle-level studies seldom report quantitative inter-wheel synchronization.

2.5. Gap Addressed by This Work

To our knowledge, no prior work combines (i) GA-tuned *bilateral cross-coupled* PID synchronization, (ii) two *mismatched*, low-cost BLDC drives, (iii) a *distributed* one-microcontroller-per-motor realization with a dedicated inter-MCU synchronization link and a custom Modbus/RS-485 command bus, and (iv) an *on-vehicle evaluation* of that combination on a 120 kg tracked platform under terrain and a one-sided load.

3. System Description and Hardware Platform

The test platform (Fig. 1) is a tracked, skid-steer differential-drive vehicle. Its two rubber tracks are driven independently by two three-phase BLDC "auto-rickshaw" motors, each through a 20 : 1 reduction gearbox. Steering comes from a difference between the two track speeds, so in straight-line motion the synchronization controller must hold the two motor speeds equal despite the motors' electrical and load mismatch. It carries a 6-degree-of-freedom manipulator arm (Fig. 1a); the assembled platform weighs 120 kg (a 75 kg carrier plus a 45 kg arm), so the tracks must coordinate



(a) Complete assembled vehicle, manipulator deployed.

(b) Carrier with the deck opened: drive electronics.

Figure 1: The tracked skid-steer differential-drive UGV. (a) The assembled platform—120 kg total, a 75 kg carrier plus the 45 kg 6-degree-of-freedom manipulator, the movable mass the two tracks must coordinate under—showing the rubber-belt tracks and their running gear (two idlers per side with a rear drive sprocket). (b) The carrier with its deck opened, identifying the drive chain: the two geared three-phase BLDC motors (Motor 1/Motor 2) at center with their off-the-shelf controllers (Driver 1/Driver 2) and per-motor microcontrollers (MCU 1/MCU 2); the dedicated inter-MCU data line; the shared Li-ion battery; the manual kill switch; the wireless receiver; and the yellow track markers (Marker 1/Marker 2) used to visualize the inter-track speed difference. The STM32F446RE master node and the RS-485 bus serving the wider vehicle are also on board.

under a substantial, movable load. Control is *distributed* across three microcontrollers, as detailed below. Platform and motor specifications are summarized in Table 1; the results of Section 6 are measured on this platform.

3.1. Mechanical and Electrical Architecture

The chassis (1.5 m×1.0 m×0.5 m) carries two rubber-belt tracks on a rigid running gear (two idlers per side, rear drive sprocket; Fig. 1a). The track gauge—the lateral distance between the track centerlines—is $B = 0.75\,\mathrm{m}$ and the ground-contact length is $\ell = 1.0\,\mathrm{m}$; this $\ell/B \approx 1.33$ ratio sets the skid-steer turning behavior. Each drive sprocket has radius $r_s = 0.20\,\mathrm{m}$.

Each track is driven by a three-phase BLDC auto-rickshaw motor (rated 48 V, 750 W, 15.6 A, four pole pairs) through its 20∶1 gearbox. The controlled variable is the motor shaft: the speed loop regulates it up to a commanded ceiling of 3000 rpm, which the gearbox converts to about 150 rpm at the sprocket and a track ground speed of $v = \omega_s r_s \approx 3.14\,\mathrm{m\,s^{-1}}$ ($11.3\,\mathrm{km\,h^{-1}}$). This gearing matters for synchronization: a relative track-speed difference Δv produces a heading rate $\dot{\psi} = \Delta v/B$, so a sustained 1% speed mismatch at full speed would integrate, uncorrected, into $\dot{\psi} \approx 2.4^\circ\,\mathrm{s^{-1}}$—about 24° of heading drift in ten seconds. This *open-loop* figure is the motivation for regulating the difference at all; once each motor tracks its own reference the residual drift is far smaller (of order a degree over a run, Section 6), and the coupling layer's role is to hold it there when a differential disturbance appears.

Each motor is driven by a ready-made, sensored auto-rickshaw BLDC controller (48 V/750 W class, current-limited near 30 A, with an internal under-voltage lockout (UVLO) near 41.5 V). The controller performs the six-step trapezoidal commutation internally from the motor's Hall sensors; the microcontroller drives it through a single analog throttle line—a pulse-width-modulation (PWM) output that an RC low-pass filter smooths to a DC voltage that sets the drive's applied level open-loop (the resulting shaft speed then depends on load and back-EMF, not a regulated set-point)—together with a direction line, and is not involved in commutation. Both motor–controller channels are identical. Power is a shared 13-cell (13S) Li-ion battery (48.1 V nominal, 15 A h, ≈720 W h): the controllers draw directly from the 48 V bus, the microcontrollers through a step-down (buck) converter. Safety is provided by a chassis-mounted manual emergency-stop that disconnects all power, and by a 20 A fuse per motor circuit (between the 15.6 A rated current and the controller's ≈30 A limit).

3.2. Distributed Control and Communication Architecture

Control is partitioned across three microcontrollers (Fig. 2) on two distinct communication paths. Each motor is regulated by its own STM32G431KB microcontroller (Nucleo-32 board, 170 MHz) running a local speed PID. The two motor microcontrollers are linked by a *dedicated point-to-point UART channel* over which each transmits its measured motor speed to the other every 1 ms using direct memory access (DMA); this fast, private link carries the data the synchronization loop needs. Separately, a master STM32F446RE microcontroller reaches both motor nodes over an RS-485 bus (MAX485 transceivers, differential twisted-pair) layered on UART at 115 200 baud with custom framing, DMA-driven transfers, and a per-packet cyclic redundancy check (CRC), distributing the operator's speed command. The two

Table 1
Platform and Motor Specifications

Parameter	Value
Vehicle	
Type	Tracked skid-steer differential drive
Mass (as tested)	120 kg (75 carrier + 45 arm)
Dimensions (L×W×H)	1.5 × 1.0 × 1.2 m
Track type / gear	Rubber belt; 2 idlers/side, rear sprocket
Track gauge B	0.75 m
Ground-contact length ℓ	1.0 m
Drive sprocket radius r_s	0.20 m
Max track speed	11.3 km h^{-1} (at 3000 rpm)
Motor and drivetrain (per side, identical)	
Motor type	three-phase BLDC (auto-rickshaw), sensored
Gearbox	20 : 1 reduction
Rated voltage / power	48 V / 750 W
Rated / no-load current	15.6 A / ≈4 A
Max commanded speed	3000 rpm (motor shaft)
Pole pairs	4
Controller	Off-the-shelf 48 V/750 W BLDC; ≈30 A limit, ≈41.5 V UVLO
Electronics and power	
Motor MCU (×2)	STM32G431KB (Nucleo-32), 170 MHz
Master MCU	STM32F446RE
Sync link	Dedicated UART, DMA, 1 ms (MCU1↔MCU2)
Command bus	RS-485 (MAX485), UART 115 200 baud; custom framing, DMA + CRC (Tanvir et al., 2024)
Data logging	Per-node serial (USB/UART) to host
Speed sensing (×2)	AMT223A-V capacitive encoder, 12-bit (4096 counts/rev), SPI; 1 kHz sampling, tracking-observer speed estimate
Battery	13S Li-ion, 48.1 V, 15 A h (shared)
Safety	Manual e-stop (all power); 20 A fuse/motor

motor nodes are two slaves on this larger multi-node bus, which also serves the vehicle's manipulator and sensors; its design, addressing, and timing are characterized in our prior work (Tanvir et al., 2024).

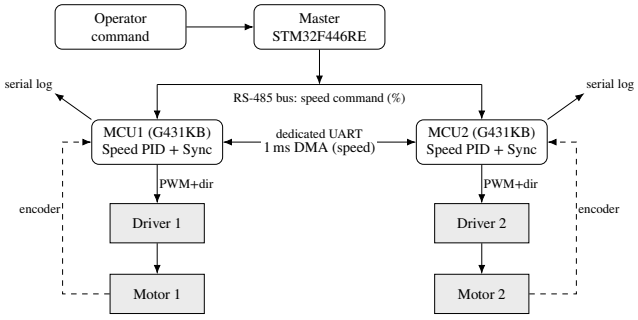


Figure 2: Distributed control architecture. The master microcontroller broadcasts a speed reference over the RS-485 bus; each motor node logs its own signals over serial. The two motor microcontrollers exchange their measured speeds over a dedicated 1 ms DMA UART link; each closes a local speed PID on its encoder-measured speed and adds a mirrored bilateral cross-coupled correction computed from both speeds, applied $\pm\frac{1}{2}$. The off-the-shelf controllers perform Hall commutation.

The master broadcasts the operator's command as a single speed reference, and straight-line motion corresponds to an equal reference on both motors, a turn to differential references. The synchronization itself is computed *locally and symmetrically on the two motor nodes*, not on the master. Each motor node maps the broadcast percentage (0–100% of maximum speed) to an rpm set-point, runs its own speed PID against its measured motor-shaft speed, and—using the other motor's speed received over the dedicated inter-MCU link every 1 ms—evaluates the bilateral cross-coupled synchronization correction. Both nodes run the identical law with the same coefficients, so both compute the same correction; each then applies one half of it with opposite sign ($+\frac{1}{2}$ on one motor, $-\frac{1}{2}$ on the other). This is a direct, distributed realization—evaluated redundantly and split $\pm\frac{1}{2}$—of the symmetric bilateral law $\beta_i = K(\omega_i - \omega_j)$ to which all two-motor coupling structures reduce (Section 2). The master only distributes commands and sequences the bus; it neither closes the synchronization loop nor logs the high-rate motor signals—each node does that over its own serial port.

3.3. Sensing and Speed Estimation

Speed feedback is taken from a contactless *capacitive* rotary encoder (CUI/Same-Sky AMT223A-V) on each motor shaft, read by the corresponding microcontroller over a private serial-peripheral-interface (SPI) bus, so the two encoders never contend. The sensor reports *absolute* shaft angle at 12-bit resolution (4096 counts per revolution); its capacitive sensing principle carries no magnet, so it avoids the once-per-revolution error a diametric-*magnet* encoder incurs from magnet misalignment. Any residual once-per-rev term is then mechanical (mounting or shaft eccentricity) and falls within the datasheet integral non-linearity of 0.2°; a full-position SPI read completes in ≈11 µs at the 2 MHz bus rate—negligible against the control period. The microcontroller samples the encoder every 1 ms; because the sampling instant is set by the microcontroller rather than by rotor events, the speed estimate arrives at a fixed, *speed-independent* rate—the property the synchronization loop of Section 5 depends on—and at the 3000 rpm ceiling the shaft advances only ≈0.05 revolution between samples, so the angle unwrap is unambiguous.

Differentiating the raw angle directly would amplify its one-count quantum ($60/(4096 \times 10^{-3}) \approx 14.65$ rpm) into unusable velocity noise, so each microcontroller instead runs a second-order critically-damped *tracking observer* on the unwrapped angle, whose velocity state is the speed estimate used by every loop. Each drive is calibrated once at commissioning for the encoder's *integral non-linearity* (INL): the deterministic, angle-dependent error that repeats

every revolution and differentiates into a periodic speed disturbance at the shaft frequency and its harmonics. This term matters precisely because the loop wants bandwidth—a wider observer passes more sensor error, so INL, not random noise, sets the usable estimate quality at ≈80 Hz. Because it is deterministic and indexed by absolute angle, it can be measured once and subtracted. The procedure is two constant-speed passes at 500 rpm over 300 revolutions ($\approx 3.6 \times 10^4$ samples): the first fits the linear angle trajectory $\hat{\theta}(t) = \hat{a} + \hat{b}\,t$ by least squares, the second bins the residuals $r_i = \theta_i - \hat{\theta}(t_i)$ into 256 equal angular bins of one revolution (≈140 samples per bin, so random noise averages down while the deterministic term converges). The table is subtracted from the measured count—in floating-point counts, never rounded, since a rounded sub-count correction differentiates into fresh speed noise—before the observer sees the angle. Calibration cuts the measured 1σ speed-estimate noise at 2000 rpm from 3.24 to 2.49 rpm (23%) at an unchanged observer bandwidth of ≈80 Hz (group delay ≈3 ms); on a second unit with larger, non-sinusoidal INL the same procedure gives 4.9 → 3.7 rpm, so the table is not restricted to a sinusoidal error shape. What survives is random noise, quantization, and speed-dependent timing effects, none of them angle-indexed—and neither are sample dropouts, which the table cannot touch: at a 0.05% dropout rate the estimate noise rises to ≈10 rpm calibrated or not, so link integrity is a precondition for the estimate quality the fast loop depends on, not an independent concern. *This observer output is the only speed the controller and the logs ever see*; the underlying raw angle is never used directly. The microcontroller reads no current or voltage telemetry—current limiting is internal to the BLDC controller, which commutates from the motor's own Hall sensors independently of this measurement—so the observer speed is the only feedback in the control loop. There is no inertial sensor (IMU, gyroscope, or magnetometer) on the vehicle.

For data collection each motor microcontroller logs its own signals (measured speed, the peer speed from the inter-MCU exchange, their difference, and the per-motor command) over its serial port; the RS-485 bus carries only the broadcast speed command. The yellow tape on the otherwise black track belts (Marker 1/Marker 2 in Fig. 1) is a qualitative visual aid for seeing a track-speed difference during a run, not a measurement instrument. Quantitative inter-track synchronization error comes from the encoder-logged speeds; the heading drift is not read from an onboard sensor (the vehicle carries none) but is *computed* from the logged track-speed difference through the differential-drive kinematics of Section 6, under a no-slip assumption that omits the track slip present on a real skid-steer surface.

3.4. Sources of Inter-Track Difference

The synchronization layer must reject any *differential* disturbance that drives one track faster than the other. Such disturbances arise from two distinct sources. The first is *electrical*: the two drives are nominally identical—the same motor, gearbox, and controller model on each side—but low-cost BLDC motors of this class vary from unit to unit in winding resistance and back-EMF constant, so at equal command one track runs persistently faster than the other. This mismatch is a structured, sustained bias (not random noise) and the one source we can partly quantify: the two deployed motors (Table 2) differ in bench-measured resistance, with an assumed back-EMF/torque spread on top, Motor 2 the lower on both, so at equal duty it runs faster (lower K_e), is slightly stronger in torque-per-duty, and therefore tends to pull ahead during hard acceleration.

The second source is *mechanical*, and for a 120 kg tracked platform frequently the larger of the two: unequal track-belt tension, differing running-gear friction and wear between the sides, and—most visibly—one-sided loads when a single track meets terrain resistance, soft ground, or an obstacle. Unlike the electrical bias, these asymmetries cannot be captured in a model a priori—they depend on assembly and drift with wear and terrain—yet they act directly on the differential (synchronization) dynamics. This is why the coupling gains cannot be tuned against a model and are instead optimized *online* on the real vehicle (Section 4). Because both sources produce the same observable—a differential track speed and the heading drift it integrates into—the same cross-coupled loop rejects both; the one-sided load of Section 6 additionally probes a severe, transient version of the mechanical case.

Table 2: Motor Parameters[a]

Parameter	Motor 1	Motor 2
Line-to-line resistance R (Ω)	0.42	0.38
Back-EMF const. K_e (V·s/rad)[b]	0.128	0.131
Torque const. K_t (N·m/A)[b]	0.128	0.131

[a]Only the resistance R is bench-measured (milliohmmeter across two motor terminals—two phases in series, hence line-to-line); the back-EMF/torque constant $K_e{=}K_t$ is a nominal value for this motor class, and its ≈8% inter-motor difference is an *assumed* unit-to-unit spread standing in for the electrical mismatch, not an independent measurement—hence "parameters," not "bench-identified." [b]Line-to-line values (two conducting phases); the per-phase constant is half. The pair is consistent with the 48 V/750 W datasheet rating. Motor 2 has the lower R and K_e, so at equal duty it runs faster and is slightly stronger in torque-per-duty. The track-side mechanical asymmetries that dominate the differential dynamics are not identifiable this way, which is why the controller is tuned online on the vehicle (Section 4).

4. Controller Design

The controller has two layers: each motor runs a local speed PID with feedforward, and above these a bilateral cross-coupled term corrects the inter-motor speed difference. Both layers are *distributed*—one microcontroller per motor—with measured speeds exchanged over a dedicated inter-MCU link so each side forms the same synchronization correction (Section 5); a separate master node broadcasts the speed command over a custom RS-485 bus of our own design (Tanvir et al., 2024). All feedback and per-motor feedforward gains are tuned online on the vehicle by a genetic algorithm (Section 4.4). Figure 3 shows the architecture: the commanded speed first passes through a second-order rate limiter producing a smooth reference r and its velocity $\dot{r}$ (peak rate $\dot{r}_{\max}$) used by the feedforward terms.

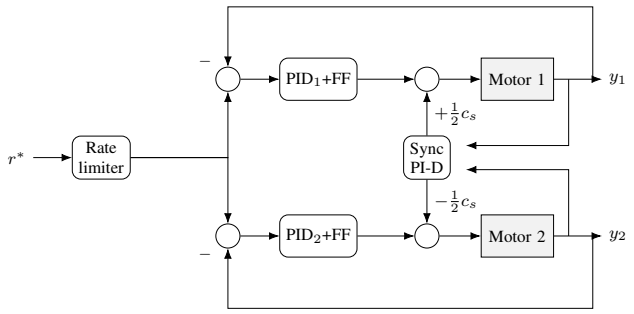


Figure 3: Distributed bilateral cross-coupled control architecture. Each motor runs a local speed PID with feedforward on its own MCU; a synchronization PI–D on the speed difference $y_2 - y_1$ (derivative taken on y_1) injects $\pm\frac{1}{2}c_s$ into the two duty commands. The rate limiter shapes the reference r.

4.1. Reference Shaping

The operator command r_{cmd} is passed through a second-order critically-damped shaper with a velocity cap, producing the smooth reference r and its rate $\dot{r}$ that the loops track and the feedforward uses:

$$\begin{aligned} &\ddot{r} = \omega_n^2(r_{\text{cmd}} - r) - 2\omega_n\dot{r}, \\ &\dot{r} \leftarrow \text{clamp}(\dot{r} + \ddot{r}\,\Delta t,\ \pm\dot{r}_{\max}), \qquad r \leftarrow r + \dot{r}\,\Delta t, \end{aligned} \tag{1}$$

with corner bandwidth $\omega_n = 5\,\text{rad}\,\text{s}^{-1}$ and slew cap $\dot{r}_{\max} = 1000\,\text{rpm}\,\text{s}^{-1}$, integrated at the fast tick $\Delta t = 1\,\text{ms}$. The cap bounds the acceleration feedforward and keeps the start-up transient within the unipolar duty range.

4.2. Per-Motor Speed PID with Feedforward

Each motor $i \in \{1, 2\}$ is regulated by a discrete PID acting on the normalized speed error

$$e_i = \text{clamp}\left(\frac{r - y_i}{y_{\max}},\ -1,\ +1\right), \tag{2}$$

with y_i the measured speed (written ω_i in Section 2 and m_i in Section 6) and $y_{\max} = 3300\,\text{rpm}$ a fixed normalization constant set just above the 3000 rpm command ceiling. The loop uses filtered derivative-on-measurement and a clamped integrator:

$$u_i^{\text{PID}} = D_{\max}\Big(K_p e_i + K_i \int e_i\,dt - K_d\,\hat{\dot{y}}_i / y_{\max}\Big), \tag{3}$$

$$\hat{\dot{y}}_i \leftarrow \alpha\,\hat{\dot{y}}_i + (1 - \alpha)\,(y_i - y_i^-)/\Delta t, \tag{4}$$

where $D_{\max} = 100\%$ duty, α is the filter coefficient and y_i^- the previous sample. The integral term is accumulated as a percent-duty quantity, $I_i = D_{\max} K_i \int e_i\,dt$, so it is directly comparable to the duty command; it is held in an *asymmetric* clamp $[-I^-,\ I^+]$ percent — a unipolar drive cannot brake, so little negative range is useful — and corrected by back-calculation anti-windup on the PID path alone,

$$I_i \leftarrow \text{clamp}\big(I_i - K_b(u_i^{\text{PID}} - \tilde{d}_i),\ -I^-,\ I^+\big), \tag{5}$$

where u_i^{PID} is the unsaturated PID output (3) and $\tilde{d}_i = d_i - u_i^{\text{FF}} - c_i$ the PID's share of the saturated command (7); removing the feedforward u_i^{FF} (6) and synchronization c_i terms makes the integrator respond only to genuine actuator saturation, not to a standing feedforward/sync offset. Two feedforward terms supply the cruise duty (plus a small acceleration term) so the integrator need not:

$$u_i^{\text{FF}} = K_{v,i}\,r + K_a\,\dot{r}, \tag{6}$$

where the back-EMF gain $K_{v,i}$ is *per motor*: each is seeded from its drive's own calibrated steady-state duty-per-speed (its inverse DC gain) and the tuner may set $K_{v,1}$ and $K_{v,2}$ independently, so each feedforward supplies its own cruise duty and the synchronization loop is left to act on genuine inter-drive differences. At this level of electrical mismatch the tuner in fact keeps the two gains nearly equal (Section 6.5), so the per-motor split is a factual per-drive convenience rather than a measured source of improvement. The duty applied to motor i is

$$d_i = \text{sat}_{[0,100]}\big(u_i^{\text{PID}} + u_i^{\text{FF}} + c_i\big), \tag{7}$$

with synchronization correction c_i (the coupling term β_i of Section 2) defined next. The duty $d_i \in [0, 100]\%$ is the throttle setpoint delivered to the off-the-shelf drive (Section 5), not a direct winding PWM. Constants: $\alpha = 0.30$, $[-I^-, I^+] = [-5, 100]\%$, $K_b = 0.04$. The filtered-derivative form follows (Romero Segovia, Hägglund and Åström, 2014); the anti-windup (5) is standard.

4.3. Bilateral Cross-Coupled Synchronization

The synchronization layer drives the inter-motor speed difference to zero using a normalized PI law on the difference error with a filtered derivative taken on y_1 (Fig. 3):

$$\begin{aligned} e_s &= \text{clamp}\left(\frac{y_2 - y_1}{y_{\max}}, -1, 1\right), \\ c_s &= D_{\max}\Big(K_p^s e_s + K_i^s \int e_s\,dt - K_d^s\,\hat{\dot{y}}_1 / y_{\max}\Big), \end{aligned} \tag{8}$$

the correction is applied *bilaterally* — split equally and oppositely between the drives:

$$c_1 = +\tfrac{1}{2}c_s, \qquad c_2 = -\tfrac{1}{2}c_s. \tag{9}$$

For two motors this bilateral law is equivalent to Koren cross-coupled control (Koren, 1980; Feng et al., 1993): each drive is corrected by half of the pairwise speed error, so neither is a fixed master (Shi et al., 2016). The proportional and integral terms act on the difference and reject a disturbance on either motor symmetrically; the derivative is taken on y_1 alone, so it is not symmetric and adds a small common-mode-to-differential term during shared acceleration—negligible under the gentle rate limit used here, but on more aggressive maneuvers a difference derivative ($\hat{\dot{y}}_2 - \hat{\dot{y}}_1$) would remove it.

The error (8) drives the speed difference to *zero*, which is the straight-line objective—equal references, r_1=r_2. A commanded turn would instead regulate the difference about its commanded value, replacing e_s with $\big[(y_2 - y_1) - (r_2 -$

$r_1)\big]/y_{\max}$; the evaluation in this paper is confined to straight-line running (Section 6.7), where the reference difference is zero and the two forms coincide.

The synchronization integrator carries its own protection, separate from the per-motor loops. Because the correction is bilateral, a negative c_s (slow the lead motor) is exactly as physical as a positive one, so the integrator uses a *symmetric* clamp $\pm I_s$ with $I_s = 50\%$—unlike the per-motor integrators' unipolar $[-5, 100]\%$ range. At start-up this integrator is initialized to a GA-tuned *preload* I_s^0 (Table 3) rather than zero, so the standing differential correction that offsets the persistent electrical mismatch is present from the first tick rather than integrated up during the initial transient. Its back-calculation anti-windup is driven by the *realized* bilateral authority rather than by the unsaturated c_s: when one drive rails at 0 or 100% duty it can no longer accept its half of the correction, so the applied differential $c_1^{\text{app}} - c_2^{\text{app}}$ falls short of c_s, and that shortfall—not the raw output—is fed back to unwind the integrator. This keeps the sync integrator honest precisely in the one-sided-load regime where a drive saturates; away from saturation the term is inert. The synchronization loop runs faster than the per-motor loops; loop rates and the inter-MCU speed exchange are detailed in Section 5.

4.4. Online Controller Tuning by Genetic Algorithm

The controller gains in (3)–(9) are tuned *online*, on the assembled vehicle, by a genetic algorithm (GA). On-vehicle tuning is a requirement here, not a convenience: the dominant inter-drive differences are *mechanical*—track-belt tension, running-gear friction, wear—which no prior plant model captures and which drift over time, so a model-tuned synchronization gain would be mistuned on the real coupled plant.

The GA evolves a twelve-gene chromosome: the three Motor-1 PID gains, the three synchronization PID gains, the synchronization-integrator preload, three gains for the co-tuned Motor-2 loop, and the two per-motor inverse-DC-gain feedforward gains $K_{v,1}, K_{v,2}$ (seeded from their calibrated values). Only the fixed structural constants (the filter coefficient α, anti-windup gains, common-mode acceleration feedforward K_a, and the integrator clamps) are hand-set; every remaining feedback and feedforward gain is GA-optimized. Table 4 lists the search bounds. Each candidate runs the vehicle through four speed-step scenarios (0→3000, 3000→1500, 1500→2500, 2500→1000 rpm), scored by fitness $f = 1/(1 + J)$ with the penalty

$$J = \frac{1}{N}\sum_{\text{scen}}\left(w_1 J_{\max} + w_2 J_{\text{tr}} + w_3 J_{\text{ss}} + w_4 J_{\text{trk}} + w_5 J_{\text{pos}}\right), \quad (10)$$

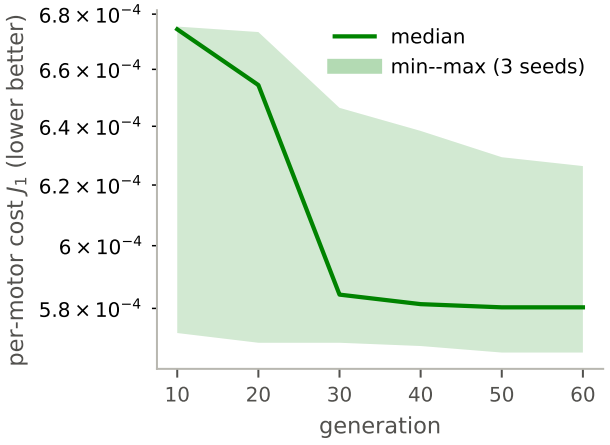
averaged over the four scenarios, where $J_{\max}$ is the worst-instant inter-motor deviation (over non-saturated samples), J_{tr} and J_{ss} the mean transient and steady-state synchronization error, J_{trk} each motor's own target-tracking error, and J_{pos} the net integrated speed mismatch (a heading-drift proxy); each term is normalized by $y_{\max}$ so J is dimensionless, and the weights are $(w_1 : \ldots : w_5) = 20 : 0.3 : 8 : 1 : 1$. The four tuning scenarios carry the platform's persistent inter-track asymmetry (Section 3) but no applied one-sided load, so the disturbance-rejection results of Section 6 lie outside the tuning set. Evolution uses simulated-binary crossover, bounded mutation with boundary escape, tournament selection, and multi-elitism, with diversity-driven re-initialization on stagnation (Hassanat et al., 2019; Joseph et al., 2022); GA-based PID tuning for BLDC drives follows (Ibrahim et al., 2019; Mahmood et al., 2025). The population is 12, tournament size 2; crossover probability is 0.8; the mutation probability is 0.3 per individual (all genes are perturbed when an individual is selected), with a 30% boundary-escape for any gene within 1% of a bound; three elites are carried each generation, and half the non-elite population is re-initialized after five stagnant generations. Every candidate is scored over the four 5 s scenarios; the scale is sized for the vehicle's real-time tuning budget rather than for offline search. *Stage 1* evolves the 12 individuals for 60 generations from each of *three* seeds ($\approx 2.2 \times 10^3$ candidate evaluations); *Stage 2* warm-starts each of the three seeds for a further 110 generations ($\approx 4.0 \times 10^3$ evaluations), so the three-seed commissioning costs $\approx 6 \times 10^3$ evaluations for that arm, of which any single deployed controller is $\approx 2 \times 10^3$; the independent-drive baseline, on the shortened plan described below, costs $\approx 4.3 \times 10^3$ over its three seeds. Every arm in the comparison of Section 6 is tuned at this cost in its own right. The objective is *stochastic*: on the real vehicle every evaluation carries its own sensor noise, ground contact, and battery state, so a given chromosome does not score identically twice. The tuner therefore re-evaluates the carried elites under fresh conditions in every generation rather than reusing a stored fitness—which is why all 12 individuals, elites included, are scored in each generation of the counts above. This implicit averaging stops a chromosome that merely drew a favourable run from persisting as an elite, at the cost of a noisier generation-to-generation trace; the convergence curves below are consequently one realization of the commissioning, not a trajectory repeatable from a seed. Both stages are run from *three independent seeds* (11, 22, 33): Stage 1 to show the per-motor tuner converges reliably, and Stage 2 to give the controller comparison of Section 6 a small distribution rather than a single point; one seed's gains are deployed.

Two-stage lab tuning. Tuning is a one-time commissioning step on the assembled vehicle, carried out in two stages that reflect what each loop needs. In *Stage 1* the two per-motor loops are tuned first, with the vehicle raised on a stand so both tracks turn freely and the coupling disabled: each motor's PID and feedforward are optimized on its *own* reference tracking—the Stage-1 objective is the tracking penalty $J_1 = \frac{1}{N}\sum_{\text{scen}} J_{\text{trk}}$ of (10) alone, the four synchronization terms $(J_{\max}, J_{\text{tr}}, J_{\text{ss}}, J_{\text{pos}})$ excluded because no gene the stage can move acts on them—independently and in parallel on the two microcontrollers. This stage needs no inter-motor differential, so the off-ground lab is ideal; its result is the warm-start for what follows. *Stage 2* re-optimizes the full

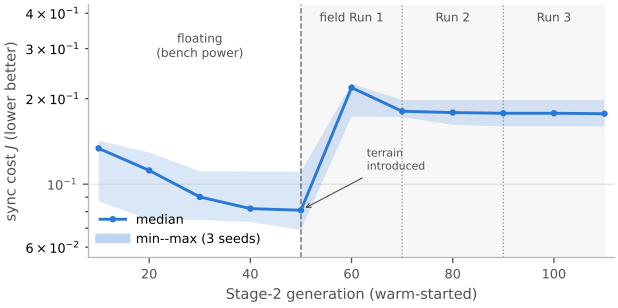
chromosome—both per-motor loops and the synchronization gains—*jointly*, warm-started from Stage 1, and is itself in two parts. *Part 1* runs on the fully-assembled but floating vehicle on bench power: the two drivetrains' mechanical asymmetry (belt tension, running-gear friction) already gives the synchronization loop a differential to act on, so its gains move off their seed without the field's endurance and real-time limits. *Part 2* then refines the controller in the field—where the terrain differential the loop is ultimately built to reject is present—as three battery-limited runs that checkpoint to flash and resume across battery swaps. The synchronization loop is a disturbance-rejection loop, so it must see a differential to tune against: floating provides a weak, mechanical-only one; the field provides the full terrain load (Fig. 4b).

The master microcontroller sequences both stages, commanding the speed-step reference scenarios; the genetic algorithm runs on the Motor-1 microcontroller, which applies its own and the synchronization genes locally and sends the Motor-2 genes to the Motor-2 node over the dedicated inter-MCU link. Because each node already receives the peer's speed every 1 ms (Section 5), both have the inter-motor error directly, and Motor-1 accumulates the per-scenario penalties of (10) into the candidate's fitness. The warm-start makes Stage 2 fast—tens of generations from a good starting point rather than a from-scratch search—so the commissioning is bounded and checkpoints to the Motor-1 flash gain store against power loss. Each motor microcontroller streams its own telemetry over its serial port to a host recorder that logs the convergence of both stages and the deployed gains; no tuning data crosses the shared command bus. Both stages are repeated from the three seeds, Stage 2 warm-starting each seed from its own Stage-1 result in the two parts above. To keep the vehicle within a short test footprint during the field part, the *common* drive direction is reversed between successive population members, so the rover rocks forward and back as the tuner evaluates candidates; both tracks always turn the same way within a candidate run, leaving the synchronization objective—the inter-motor *difference*—unchanged.

The independent-drive baseline evaluated in Section 6 is tuned by this same harness with the synchronization term disabled throughout, on a *shortened* plan: Stage 1 off-ground as above, then—skipping the floating Part 1, which exists to give the synchronization loop a mechanical-only differential a controller with no synchronization gains cannot use—directly to the three battery-limited field runs of Part 2, scored on the full objective (10) with the four synchronization genes inactive and the remaining eight free. It is therefore a field-tuned independent pair that has met the same operating differential and been scored on the same inter-motor penalties as the coupled arms—the strongest independent PID this harness produces, not a Stage-1 point carried forward—but over a shorter search: 60 Stage-1 plus 60 field generations against the coupled arms' 60 plus 110. What it cannot do is *act* on the difference directly; its per-motor loops can only reduce the inter-motor error by tracking their common reference more stiffly.



(a) Stage 1: per-motor loops, floating, on the per-motor tracking cost J_1.



(b) Stage 2 (three seeds): sync tuned first on the floating plant (bench power), then refined in the field over three battery-limited runs; cost steps up when the terrain differential is introduced, the first field run recovers about a quarter of the step, and the cost is flat thereafter at roughly twice the floating value.

Figure 4: Two-stage tuning convergence (reduced on-vehicle plan): the *best-so-far* GA cost J (the penalty of (10), $J{=}1/f - 1$; lower is better, log axis), median and min–max band over the three seeds in both panels. The incumbent record is monotone within each phase by construction; the underlying per-generation scores are not, because the on-vehicle objective is stochastic and every candidate—elites included—is re-evaluated under fresh conditions (Section 4.4). The two panels minimize different objectives (J_1 vs the full J), so their cost scales are not comparable.

Stage 1 converges within tens of generations on the floating per-motor plant (Fig. 4a); the three seeds reach a consistent band with differing gain *vectors*—the objective is nearly flat in gain space, so many gain sets tune about equally well (the feedforward gains $K_{v,i}$ are the exception, pinned to each drive's inverse DC gain). Stage 2 (Fig. 4b) first drives the synchronization cost down on the floating plant, then absorbs a step increase when the terrain differential is introduced in the field; the first field run recovers roughly a quarter of that step and the cost is flat thereafter, settling at about twice the floating-plant value—the terrain differential is a harder problem than the mechanical-only one, and the field runs do not tune it away. Table 3 lists the gains of the deployed seed, frozen and evaluated in Section 6; across the

three seeds the reduced on-vehicle plan reaches a synchronization fitness of ≈0.85 (min–max band, Fig. 4b), and the same three seeds carry through to the held-out evaluation and the controller comparison of Section 6.

Table 3: Representative GA-Tuned Controller Gains

Loop	K_p	K_i	K_d
Motor 1 speed	1.775	8.015	0.00010
Motor 2 speed	1.732	6.225	0.00473
Synchronization	3.636	18.83	0.02325
Sync integrator preload = 0.0667			
Feedforward: $K_{v,1} = 0.0258$, $K_{v,2} = 0.0265$, $K_a = 2.0 \times 10^{-4}$			

The gains of the deployed seed (one of three Stage-2 runs). K_a is a fixed common-mode constant, not a GA gene. The objective is nearly flat in gain space, so the feedback gains are weakly determined—the synchronization K_p^s alone spans ≈0.6–3.6 across the three seeds—while the feedforward gains $K_{v,i}$ are tight (≈5%), pinned to each drive's inverse DC gain.

Table 4: GA Gene Search Bounds

Gene(s)	min	max
Per-motor K_p (M1, M2)	0.01	5.0
Per-motor K_i (M1, M2)	0.05	15.0
Per-motor K_d (M1, M2)	0.0001	0.05
Synchronization K_p^s	0.01	20.0
Synchronization K_i^s	0.1	50.0
Synchronization K_d^s	0.0001	0.75
Sync integrator preload	0.01	2.0
Feedforward $K_{v,1}$, $K_{v,2}$	0.024	0.032

5. Embedded Implementation

This section describes how the controller of Section 4 runs on the vehicle.

5.1. Distributed Firmware Partition

Control is split across three STM32 microcontrollers. Each motor is driven by its own STM32G431KB (Nucleo-32, 170 MHz) that closes the local speed loop of (2)–(7) and computes the bilateral synchronization correction of (8)–(9). A master STM32F446RE converts the operator's motion command into a speed reference and distributes it. Being symmetric, the synchronization law is evaluated *identically and independently on both motor nodes* rather than on the master: each node receives the other motor's measured speed, forms the same correction c_s, and applies its half with the appropriate sign ($+\frac{1}{2}c_s$ on Motor 1, $-\frac{1}{2}c_s$ on Motor 2). Neither motor node is master of the other, and the F446RE never closes the synchronization loop.

The motor microcontrollers do not perform commutation. Each drives an off-the-shelf sensored BLDC controller (Table 1) over a single analog throttle line and a direction line; the controller performs the six-step Hall commutation and enforces its own current limit. The microcontroller's role is to compute the duty command $d_i \in [0, 100]\%$ of (7) and deliver it as a throttle voltage: a PWM output is low-pass filtered by an RC network to a DC level, so the duty maps directly to the analog throttle voltage, which sets the drive's applied level open-loop (the resulting shaft speed then depends on load).

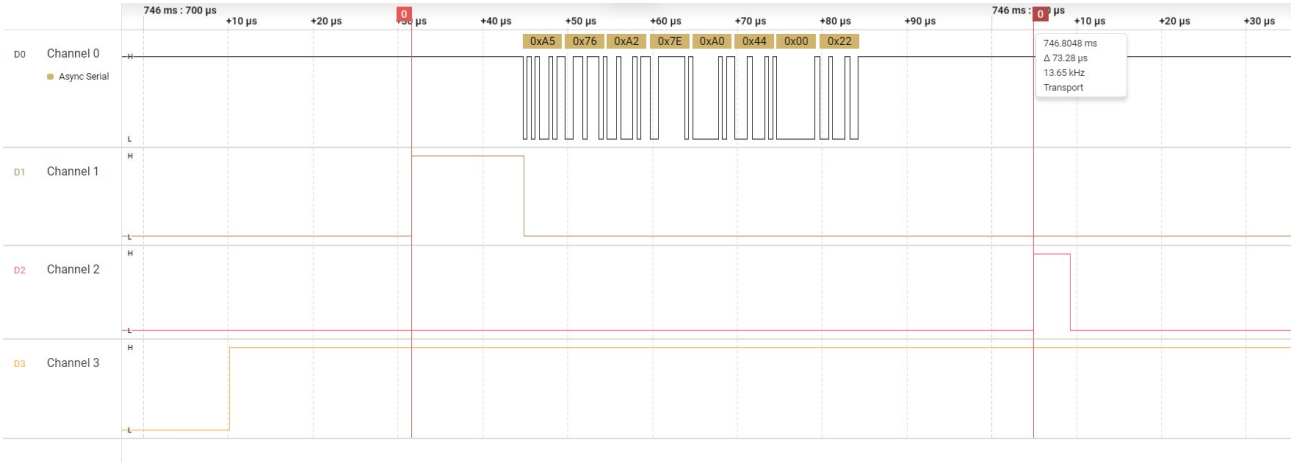
5.2. Communication Links

Two physically separate links serve two different needs (Fig. 2). The *synchronization* link is a dedicated point-to-point UART between the two motor microcontrollers: each transmits its latest measured speed to the other every 1 ms using DMA, so each node always has a fresh peer speed for (8). This private, deterministic channel lets the synchronization loop run at the rate the differential disturbance demands. Each exchange is a fixed 8-byte frame—a start delimiter (`0xA5`), a rolling sequence counter, the single-precision (IEEE-754, little-endian) measured speed, a status byte, and a CRC-8 (polynomial `0x07`)—sent at 2 Mbaud, 8N1; the 170 MHz clock divides to exactly 2 Mbaud, so there is no baud-rate error. Transmission is DMA-driven (normal mode) and reception uses a circular DMA buffer with UART idle-line events, so a dropped byte cannot desynchronize the stream and the receiver re-locks on the next start delimiter. The resulting transport latency, from sampling the speed on one node to its availability on the other, is bench-measured at ≈75 µs (Fig. 5, Table 5)—40 µs of wire time plus ≈35 µs of software (DMA setup, idle-line detection, the CRC check, and the receive interrupt)—about 7.5 % of the 1 ms exchange period. The budget is therefore dominated by the exchange *hold*: because the two 1 ms loops are not phase-locked (their free-running clocks differ by ≈0.2 %), a received peer speed is at most one period old (≤1 ms, mean 0.5 ms) when the control loop reads it—the half-sample delay already carried in τ_d of the loop-transfer analysis (Section 6.5). A sequence gap or CRC failure drops the frame and holds the last valid speed; if no valid frame arrives for more than 5 ms (five exchange periods) the node zeroes its synchronization correction and reverts to independent per-motor control, so a lost link degrades gracefully rather than injecting a stale differential. The *command* link is a separate RS-485 bus (MAX485 transceivers, UART at 115 200 baud, custom framing, DMA transfers, per-packet CRC) on which the master broadcasts the speed reference—a single percentage of maximum speed—to both motor nodes. It carries only the set-point, not high-rate telemetry, so its bus load is negligible. The motor nodes are two slaves on this same multi-node bus, which also serves the vehicle's manipulator and sensors; its protocol, addressing, and measured timing are reported in our prior work (Tanvir et al., 2024). Separating the two links keeps the slower, shared command bus out of the synchronization path.

5.3. Control-Loop Timing

The firmware runs two control loops at different rates. The per-motor speed PID with feedforward updates every 5 ms (200 Hz), while the synchronization correction—and the duty command sent to each controller—updates every 1 ms (1 kHz), matched to the inter-MCU speed exchange. The five-fold gap is deliberate: the per-motor loops track a smooth, rate-limited reference and need only modest bandwidth, whereas the inter-motor *difference* can develop

(a) One 1 ms exchange period.



(b) One frame decoded; 73 µs transport marker.

Figure 5: Inter-MCU synchronization link on a logic analyzer (2 Mbaud, 8N1). Traces, top to bottom: USART TX (channel 0, async-serial decode), `TP_TX` (channel 1, speed-sample / transmit instant), `TP_RX` (channel 2, CRC-valid peer frame published), `TP_TICK` (channel 3, 1 kHz tick). Decoded bytes, left to right: SOF (`0xA5`), SEQ, speed (`float32` LE, 4 B), FLAGS, CRC-8; the frame shown is valid (SEQ 118, 1284.0 rpm, CRC `0x22`). The 40 µs frame is a small fraction of the period, so the asynchronous *hold* dominates latency; the marked transport is 73 µs (74.8 µs mean, Table 5).

Table 5
Measured inter-MCU synchronization link performance.

Parameter	Value
Baud / format	2 Mbaud, 8N1 (exact BRR)
Frame / wire time	8 B / 40 µs
Exchange rate	1 kHz, jitter <1 µs
Transport latency	74.8 µs (≈7.5 %)
Exchange hold	0 ms to 1 ms, mean 0.5 ms
Inter-node clock offset	≈0.2 % (free-running)
Integrity (soak)	0 CRC/seq err, 5.66×10^{6} frames
Link-loss behavior	stale <5 ms, corr. zeroed

abruptly—for instance when one track meets a load—and the encoder supplies a fresh absolute-angle sample every 1 ms at a fixed rate that does not fall away as shaft speed drops. Matching the difference loop to that speed update lets it counteract a developing lead/lag before the duty command saturates, whereas a 5 ms loop would lag it. This multi-rate cadence proves the dominant lever for transient and disturbance-rejection performance (Section 6): at a single loop rate the linear coupling gain must be traded down for stability margin, and its disturbance-rejection bandwidth falls with it; the sliding-mode law gains nothing from a faster rate. The drive is unipolar, so each duty command is saturated to $[0, 100]\%$ (7); because the controller cannot brake, the integrator uses the asymmetric clamp and back-calculation anti-windup of (5). The high-frequency phase-commutation switching is generated inside the off-the-shelf controllers and is not a firmware parameter.

5.4. Sensing, Command, and Safety

Speed feedback is derived from the per-motor capacitive encoder (Section 3): the microcontroller reads the absolute shaft angle over SPI every 1 ms and passes the unwrapped angle through a second-order tracking observer, so the speed estimate is refreshed at a fixed rate set by the firmware rather than by rotor events. No current or voltage telemetry is read—current limiting is internal to the controller—so the observer speed is the only feedback in the loop, and there is no inertial sensor on the vehicle.

The command chain is: the operator issues a motion command (forward, reverse, or a turn); the master maps it to a speed reference as a percentage of maximum speed and broadcasts it over the RS-485 bus; each motor node converts the percentage to an rpm set-point (0 rpm to 3000 rpm at the motor shaft) and tracks it with its local loop plus the synchronization correction. A straight-line command—equal references to both motors—is the case the synchronization layer is designed for. Safety is handled by a chassis-mounted manual emergency stop that disconnects all power and by a 20 A fuse per motor circuit.

5.5. Data Logging

During each run each motor microcontroller streams its own log—its measured speed, the peer speed it receives over the 1 ms exchange, their difference, and its duty command—over its own serial (USB/UART) port to a host recorder, so the high-rate logs never load the RS-485 command bus. Because each node already holds both motor speeds (its own and the peer's, sampled one exchange period apart), the inter-track synchronization error is read directly from a single node's log without cross-node alignment. These logged signals are the quantitative measurement used for the results of Section 6; the yellow track markers (Section 3) serve only as an in-the-field visual check and are not used quantitatively. The same per-node serial path also carries the tuning-session telemetry of Section 4.4—the per-generation best fitness and current gains—and the deployed gains themselves persist in a CRC-checked on-chip flash key–value store, so a tuned controller survives power cycles and an interrupted tuning run resumes from its last checkpoint rather than restarting.

6. Results

The controller gains are tuned *online* on the assembled vehicle (Section 4); the four tuning scenarios carry the platform's persistent inter-track asymmetry but no applied load disturbance. We then freeze the gains and evaluate the closed loop on a *held-out* test set. By *held-out* we mean out-of-training: the genetic algorithm optimized the gains against the four tuning steps $0 \rightarrow 3000 \rightarrow 1500 \rightarrow 2500 \rightarrow 1000$ rpm, whereas every tracking scenario reported below runs the distinct steps $0 \rightarrow 2800 \rightarrow 800 \rightarrow 2200 \rightarrow 1200$ rpm, and the load disturbance was never part of tuning—so no result here is a replay of the tuning conditions. Nor is the ground replayed: each evaluation run traverses the course afresh, so every held-out run meets terrain the Stage-2 tuner never optimized against, though it remains the same course (a robustness sweep over distinct surfaces is left to future work; Section 6.7).

Two conventions hold throughout. First, every reported speed is the on-board *observer estimate* (Section 3.3)—exactly what each microcontroller computes and logs—not the raw differentiated angle and not an idealized speed; a one-count quantum is $60/(4096 \times 10^{-3}) \approx 14.65$ rpm, so no *instantaneous* difference below the estimator's measured $\approx$2.5 rpm noise floor (Section 3.3) is claimed. That floor bounds a single sample, not a statistic: every metric below is an RMS, mean or peak taken over $\sim 10^4$ samples per run and then over three runs, and averaging drives the uncertainty of a reported mean well beneath the per-sample floor—the clean-tracking RMSE run-to-run SDs are 0.5 rpm to 1.8 rpm, smaller than the 2.5 rpm sample noise they are computed from. What limits the resolution of a reported *mean* is therefore that run-to-run SD, not the sensor, and a margin narrower than 2.5 rpm between two means is resolved, or not, by the SD quoted beside it. We report both throughout. Second, because the tuner is stochastic, each controller is tuned from *three independent GA seeds* (11, 22, 33)—the reduced on-vehicle plan of Section 4.4—and every value is reported as mean $\pm$ standard deviation over those three. We mark two means with * when they differ by more than twice the pooled standard deviation, but with only three seeds this is a coarse *effect-size* indicator, not a hypothesis test, and we read the results as trends and rankings rather than resolved separations throughout. Each of the three is a physically distinct run on the same test course, so—unlike a seeded simulation—this spread is not tuner stochasticity alone: it bundles the tuner's own variability with genuine run-to-run variation in ground contact, battery state, and sensor noise. It is therefore a *repeatability* figure rather than a decomposition, wider than a tuner-only spread would be—so a separation that clears it clears a stricter bar—but correspondingly unable to attribute a difference to either source (Section 6.7). Because every arm is evaluated on the same three seeds, we additionally report a paired robustness check—the per-seed difference d = base − ours and its own standard deviation—and flag any separation that clears the unpaired bar but not the paired one, $|\overline{d}| > 2\,\mathrm{SD}(d)$ (Section 6.4).

6.1. Metrics

From the observer speeds m_1, m_2 (motor-shaft rpm) we report: the *instantaneous synchronization error* $m_1 - m_2$; the *steady-state* error (mean $|m_1 - m_2|$ over the settled second half of each scenario); the *peak transient* error (maximum $|m_1 - m_2|$); the root-mean-square synchronization error (*RMSE*) over the run; and the *heading drift*, a *kinematic estimate* obtained by integrating the track-speed difference through the differential-drive kinematics of Section 3 under a no-slip assumption,

$$\psi(t) = \frac{180}{\pi} \int_0^t \frac{(m_1 - m_2)\,\frac{2\pi}{60}\,r_s}{N\,B}\,d\tau, \tag{11}$$

with gear ratio $N = 20$, sprocket radius $r_s = 0.20$ m, and track gauge $B = 0.75$ m. It isolates the heading error attributable to track-speed mismatch; a real skid-steer surface adds track slip that this idealized relation omits, and—because the encoder measures the motor shaft, not the track—any drive-sprocket radius mismatch is invisible to it, so (11) is a lower bound on real heading error.

6.2. Synchronization ON vs. OFF

Figure 6a shows the inter-motor difference over the held-out tracking run for the proposed controller (*ON*) against the independent-drive baseline (*OFF*): a single speed PID

with per-motor feedforward and the synchronization loop disabled, itself independently GA-tuned, so the architectural difference between the two is the coupling layer. The traces are broadband under the test surface, so we overlay a 0.5 s running RMS. Across the run the coupling loop holds the inter-motor difference near its noise floor while the uncoupled pair rides higher through every step transition; Table 6 gives the held-out statistics over the three seeds.

Synchronization cuts the held-out clean-tracking RMSE by 41% (10.67 → 6.26 rpm, 3.9σ; paired 2.1)—the one reduction here that clears the 2σ bar and also survives the paired test. The largest single improvement is on the one-sided load, a 65% cut in sustained error (24.9 → 8.7 rpm), and the clean-tracking peak falls 38% (83.6 → 51.5 rpm); but at 1.4σ and 1.9σ these are consistent trends rather than separated reductions, and pairing by seed does not rescue the load result (paired 1.2, weaker than the unpaired figure). The load response and the cold-start peak both vary widely from tuning to tuning, so these are reductions in the mean rather than margins each tuning is guaranteed to deliver. Heading is now a positive but modest story. Accumulated heading is a random walk, so the honest quantity is its seed-to-seed *spread*: the uncoupled drives fan out (magnitude 1.23 ± 1.14°; min–max envelope −2.0 to +3.5°, Fig. 6b) while all three coupled seeds stay within a narrower band (0.84 ± 0.55°; envelope −1.6 to +0.8°)—a 38% tighter heading random walk (RMS 1.54 → 0.95°; 0.4σ on the mean), in the intended direction: the difference-nulling loop that flattens Fig. 6a also bounds the heading it integrates into.

Table 6: Synchronization ON vs. OFF on the Held-Out Set (mean ± SD over three GA seeds; * exceeds 2 pooled SD)

Metric	OFF	Ours	Impr.
Clean-tracking RMSE (rpm)	10.67 ± 1.24	6.26 ± 1.03	41%*
Clean-tracking peak (rpm)	83.6 ± 15.5	51.5 ± 17.6	38%
One-sided load (rpm)	24.9 ± 15.8	8.7 ± 2.8	65%
Heading drift (deg)	1.23 ± 1.14	0.84 ± 0.55	32%

Clean tracking is the held-out four-step run; the ≈80 kg one-sided load is a held-out cruise disturbance at 2000 rpm. Only clean RMSE clears the 2σ bar and survives the paired test; the clean-peak, load, and heading rows are consistent trends but do not clear it at three seeds.

6.3. Disturbance Rejection

Figure 7 applies a held-out one-sided load while the vehicle cruises at 2000 rpm: for about 2 s (t=4–6 s) an ≈80 kg load is carried off-centre, its weight bearing on one side of the deck clear of the track, on top of the platform's persistent inter-track asymmetry. This is the everyday case the synchronization layer exists for—cargo mounted or shifted to one side—rather than a contrived actuator fault. The vehicle carries no torque or force sensor (Section 3.3), so the disturbance is characterized by its mass and the side it bears on rather than by an applied torque, and it is deliberately not a clean step: applying and releasing the load each inject their own transient, which is why the difference recovers only after a second, smaller excursion once the load comes off. The disturbance is one-sided, so its *differential* part drives the tracks apart; both motors retain duty headroom at this speed, so it is rejectable rather than saturating. Without coupling the loaded track sags below the other, holding a sustained difference of ≈25 rpm across the three seeds; the bilateral loop shares the correction across both drives and cuts it to ≈8.7 rpm (65%, Table 6), the largest single improvement we measure. This is the mechanical-asymmetry case—an off-centre payload, a track meeting terrain resistance, or a tension or wear difference—for which the synchronization layer exists on a heavy tracked platform.

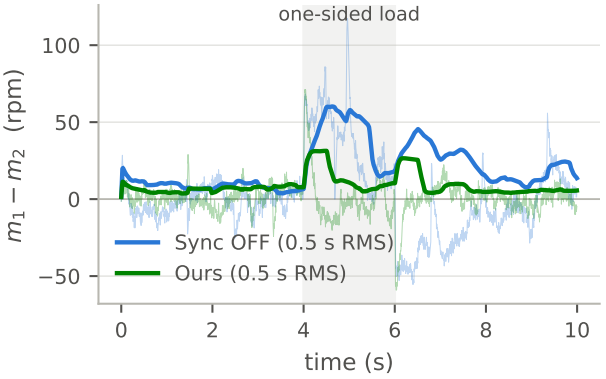
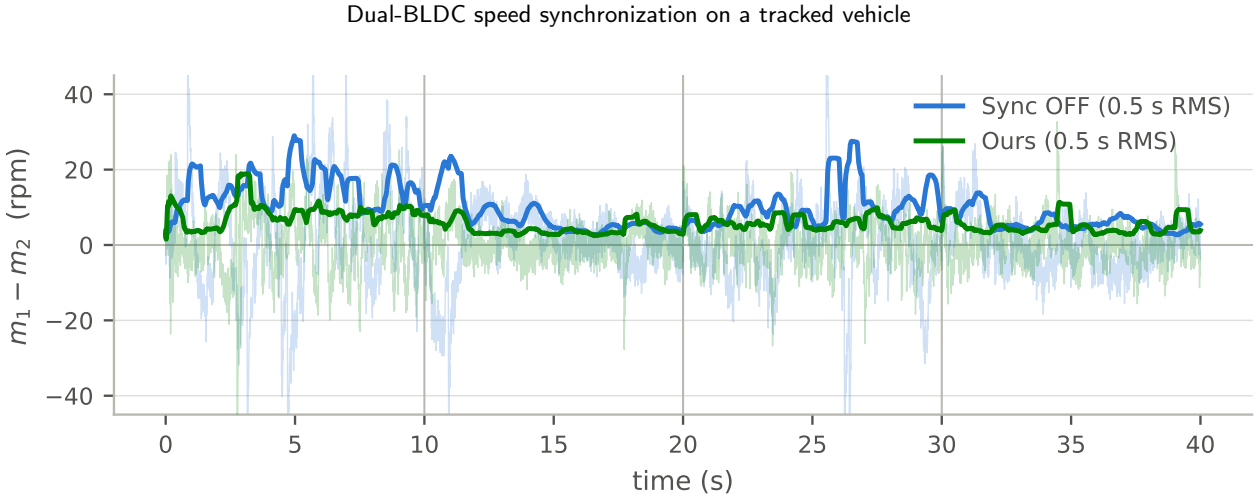


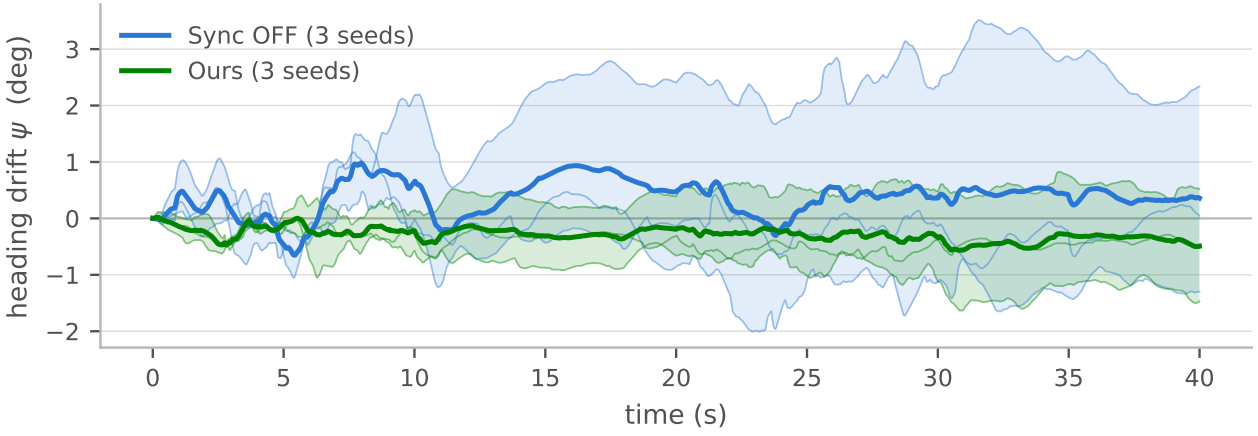
Figure 7: Held-out ≈80 kg one-sided load—an off-centre mass bearing on one side of the deck—at 2000 rpm cruise (t≈4–6 s, shaded), representative seed, 0.5 s running RMS. Under the load the uncoupled pair's difference climbs steeply while the coupling loop holds it far lower: in the shaded window the seed shown holds ≈12 rpm with coupling versus ≈35 rpm without, and across the three seeds the window-mean difference is 8.7 rpm versus 25 rpm (Table 6).

6.4. Controller Comparison on the Same Plant

That synchronization helps does not establish which controller to use. We compare four on the *identical* plant, operating points, and disturbances: (i) independent PID (synchronization off); (ii) conventional cross-coupling (Koren, 1980), a bilateral PID synchronization law; (iii) a cross-coupled boundary-layer super-twisting sliding-mode law (Levant, 1993), a strong modern baseline; and (iv) our multi-rate controller. All four arms—including the independent-PID baseline (i)—are tuned to convergence by the same genetic algorithm (same population, scenarios, and fitness), each in its own configuration; baseline (i) is therefore the best independent PID—the same harness with the four synchronization genes inactive and the remaining eight free, run off-ground and then over the three field runs (Section 4.4)—not the coupled gains with the loop merely switched off. Its plan is shorter than the coupled arms' by the floating Part 1, which carries no signal for a controller with no synchronization gains to move; the baseline therefore searches 120 generations against the coupled arms' 170, an asymmetry that favours the coupled arms and which we do not net out of the ON-vs-OFF margins below. Baselines (ii),(iii) are *single-rate*, as these laws are applied in the literature; the proposed controller (iv) adds *per-motor* inverse-DC-gain feedforward (Section 4) and the 1 kHz *multi-rate* synchronization loop (Section 5)—the latter the mechanism we contribute—whereas the conventional baselines use a single



(a) Held-out run (0 → 2800 → 800 → 2200 → 1200); light traces the raw difference, bold the 0.5 s running RMS.



(b) Accumulated heading drift, all three seeds (per-seed traces, min–max envelope, and mean).

Figure 6: Held-out results, synchronization ON (proposed) vs OFF (independent drive), on operating points the tuner never saw. The representative seed in (a) is the one nearest the median clean-tracking RMSE; (b) shows all three seeds. (a) Inter-motor difference $m_1 - m_2$; the coupling loop holds the running-RMS difference near the estimator floor while the uncoupled pair rides higher through every transition. (b) Accumulated heading drift (11); heading is a random walk, and all three coupled seeds stay within −1.6 to +0.8° while the uncoupled drives fan out over −2.0 to +3.5°—a tighter heading spread (Table 6), a trend at three seeds.

shared feedforward. The Ours-vs-CCC comparison therefore nominally combines two changes—the loop rate and the per-motor feedforward—but Table 8 shows the two laws are indistinguishable at a matched rate (§6.5), so in practice the whole difference is the loop rate. Both baselines and our controller are our own same-plant re-implementations tuned by the identical harness—a controlled comparison, not a transcription of the source papers' hardware numbers. Table 7 reports mean ± SD over three seeds and Fig. 8 plots it.

The super-twisting arm replaces the linear law (8) with a boundary-layer second-order sliding-mode controller on $\sigma = y_2 - y_1$:

$$c_s = k_1\,|\sigma|^{1/2}\,\mathrm{sat}\big(\sigma/\Phi\big) + z, \qquad \dot{z} = k_2\,\mathrm{sat}\big(\sigma/\Phi\big), \quad (12)$$

with reaching gains k_1, k_2 GA-tuned in the same gene slots as the linear sync gains and a boundary layer $\Phi = 4$ rpm that softens the discontinuous sign(·) so estimator noise does not chatter. Replacing sign by $\mathrm{sat}(\sigma/\Phi)$ sacrifices the finite-time convergence and exact rejection of the ideal super-twisting algorithm, so (12) is properly a smooth boundary-layer approximation; we retain it as a strong nonlinear baseline. The output is split bilaterally exactly as in (9).

Three findings stand out. First, *coupling helps most on disturbance rejection, and the multi-rate loop is what helps on tracking*. With every coupled arm sharing the same per-motor base (Section 4.4), the multi-rate controller and the super-twisting law clearly beat no synchronization on held-out clean tracking (RMSE 6.26 rpm at 3.9σ and 6.61 rpm vs the no-sync 10.67 rpm), while *single-rate* cross-coupling does not—it is in fact marginally worse than no synchronization (11.44 rpm) on this metric. On the one-sided load, by contrast, *every* coupled arm cuts the sustained error sharply (≈11 rpm for the single-rate laws and 8.7 rpm for ours, against the no-sync 24.9 rpm)—rejecting a one-sided disturbance is the coupling layer's purpose, though

the wide run-to-run spread on this metric leaves the size of the margin unresolved at three seeds. The multi-rate loop then stands out on the *transient* peak—a 38% reduction (83.6 → 51.5 rpm, a 1.9σ trend)—where the single-rate loops do not help: CCC is actually worse than no-sync (93.8 vs 83.6 rpm) and SMC only 17% better. A single-rate coupling loop carries too little phase-margin headroom to arrest a fast transient and can even inject peak error, as CCC does here. Second, against conventional single-rate cross-coupling the proposed controller reduces clean-tracking RMSE by 45% and clean peak by 45% (unpaired 3.6 and 2.0σ), with a 20% one-sided-load reduction. This gap is *entirely* the loop rate, not the coupling law: at a matched 1 kHz rate conventional cross-coupling and our controller are indistinguishable *on RMSE* (6.58 vs 6.26 rpm, within their overlapping spread, Table 8), so the per-motor feedforward adds nothing measurable here—the whole gap is single-rate cross-coupling (11.44 rpm) becoming multi-rate (6.58 rpm). On the transient *peak* the matched-rate comparison does not merely tie but runs the other way: multi-rate CCC reaches 35.2 rpm against our 51.5 rpm (Table 8). At 1.2 pooled SD that is not a separation at three seeds, but it is not evidence for the proposed law either— so the peak advantage we report over *single-rate* cross-coupling is a property of the rate, and does not carry over to a rate-matched conventional loop. The controller we contribute is therefore best read as conventional cross-coupling *made multi-rate*; none of these separations survives the paired test (1.9 RMSE, 1.1 peak, 0.4 load, below). Third, against the boundary-layer super-twisting baseline the proposed controller essentially *matches* it on clean-tracking RMSE (6.26 vs 6.61 rpm, 0.4σ) and leads on the transient peak (26%) and on the one-sided load (22%), though every super-twisting lead margin sits within run-to-run spread ($\leq$1.3σ, paired and unpaired). Our transparent law thus matches a strong nonlinear baseline without a sliding surface or chattering mitigation.

Paired robustness. Because each arm is independently re-tuned on every seed, pairing by seed need not tighten the between-controller contrast; with only three seeds the per-arm correlation is not meaningfully estimable, so we report the paired effect size $\overline{d}/\mathrm{SD}(d)$ alongside the unpaired one and lean on whichever is weaker. The synchronization-on-vs-off gain survives the paired test on the one metric that also clears the unpaired bar (paired 2.1 clean RMSE; Table 6). Among the coupled arms nothing separates: against conventional cross-coupling the paired figures are 1.9 (RMSE), 1.1 (peak), and 0.4 (one-sided load), and against the super-twisting law 0.6–1.3 across every metric. The controller comparison is therefore a consistent *ranking*—Ours has the lowest mean on every metric—not a resolved separation from either coupled baseline.

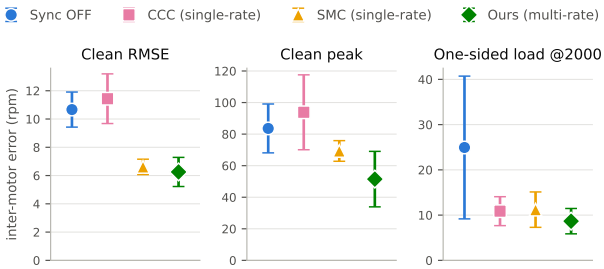


Figure 8: Same-plant controller comparison, mean ± SD over three GA seeds, by metric (each arm a distinct colour and marker; "Ours" in green as in Fig. 6). Conventional cross-coupling (CCC) and the boundary-layer super-twisting SMC are single-rate; the proposed controller is multi-rate. The proposed controller is lowest on every metric; every coupled arm cuts the sustained one-sided-load error, but single-rate CCC is worse than no-sync on clean RMSE and peak, and only the multi-rate loop reduces the transient peak (a 1.9σ trend).

Table 7: Same-Plant Controller Comparison (held-out set, mean ± SD over three GA seeds; conventional laws single-rate, proposed controller multi-rate; lowest per column in bold)

Controller	Clean RMSE (rpm)	Clean peak (rpm)	One-sided load (rpm)
Independent PID	10.67 ± 1.24	83.6 ± 15.5	24.9 ± 15.8
CCC (single-rate)	11.44 ± 1.76	93.8 ± 23.7	10.9 ± 3.2
SMC (single-rate)	6.61 ± 0.54	69.3 ± 6.5	11.2 ± 3.9
Ours (multi-rate)	**6.26 ± 1.03**	**51.5 ± 17.6**	**8.7 ± 2.8**

Independent PID is the no-sync baseline. Conventional cross-coupling (CCC) and super-twisting (SMC) are our own same-plant re-implementations, tuned by the identical GA harness. Clean tracking is the held-out four-step run; the ≈80 kg one-sided load is a held-out cruise disturbance at 2000 rpm. All arms report the observer estimate.

6.5. Loop Rate: Single-Rate vs. Multi-Rate

The gap between the single-rate baselines and the multi-rate proposed controller in Table 7 raises the question of how much is the coupling law and how much the loop rate. Re-tuning each coupling law at both rates by the same GA (Table 8) isolates it. Running the synchronization loop at 1 kHz rather than 200 Hz improves conventional cross-coupling decisively (clean RMSE 11.44 → 6.58 rpm, 2.6σ; peak 94 → 35 rpm, 3.3σ—the only rate effect in this table that clears the 2σ bar), moves our controller the same way but within spread (RMSE 7.66 → 6.26 rpm, 0.9σ; peak 65 → 51 rpm, 0.7σ), and leaves the super-twisting law flat (RMSE 6.61 → 6.05 rpm, 0.5σ; peak unchanged within spread). The rate effect is thus established on the conventional law and is a consistent trend, not a separation, on ours. Because each law is re-tuned *independently* at each rate, the two single-rate rows (CCC 11.44 rpm, Ours 7.66 rpm) differ in all twelve genes, not only the feedforward—their feedforward gains are in fact nearly identical (≈0.028 %/rpm on both drives)—so the ordering between them is a difference of loosely-determined coupling gains within tuner spread (single-rate CCC even overlaps no-sync at 11.44 ± 1.76 rpm), not a feedforward penalty. At 200 Hz the tuner drives the coupling gain low for want of the stability margin that would reward it—the same loop-rate lever restated—so the robust reading

of Table 8 is the *within-law* rate gain, CCC improving decisively and our controller consistently but within spread, from 200 Hz to 1 kHz. The super-twisting law does not share this sensitivity: its boundary layer $\Phi = 4$ rpm is fixed by the estimator noise floor rather than the loop period, so a faster tick does not raise its disturbance-rejection bandwidth the way it lifts the linear coupling's crossover; the fast rate barely moves its RMSE and slightly worsens its peak, both within a large seed spread. The reason the fast loop pays off here—where a naive fast loop did not in earlier iterations of this work—is feedback latency: the tracking observer's group delay falls to ≈3 ms at the calibrated ≈80 Hz bandwidth (Section 3.3), a few 1 kHz ticks, so the fast loop can correct a developing lead/lag before it grows; a lower-bandwidth estimate adds lag comparable to the loop period itself and erases the advantage. The useful synchronization loop rate is thus bounded by the speed-estimate latency, which is set by the sensor—a bound the following loop-transfer analysis makes quantitative.

A loop-transfer view. For two near-identical drives the synchronization layer acts on the inter-motor difference through the open-loop transfer

$$L(s) = C_s(s)\, P_\Delta(s)\, G_o(s)\, e^{-s\tau_d}, \tag{13}$$

where $C_s(s)$ is the normalized synchronization PI–D of (8) with the Table 3 gains; $P_\Delta(s) = K_\Delta/(\tau_\Delta s + 1)$ is the difference-mode duty-to-speed response identified at 2000 rpm cruise ($K_\Delta \approx 35.8$ rpm per % duty, $\tau_\Delta \approx 25$ ms); $G_o(s) = \omega_n^2/(s^2+2\zeta\omega_n s+\omega_n^2)$ is the 80 Hz tracking observer (ζ=1); and $\tau_d = T_c/2 + T_{\text{enc}}/2$ lumps the control zero-order hold at the loop period T_c and the 1 kHz sensor sample. (The per-motor loops regulate the common mode; the difference mode is approximated by the open per-motor response, and the synchronization derivative, taken on y_1 in firmware, is modeled here on the difference.)

At the deployed gains the 1 kHz loop crosses over at $f_c \approx 39$ Hz with a comfortable phase margin of ≈86° and gain margin ≈10 dB for the representative seed (Fig. 9), well within the bandwidth the 80 Hz observer allows. Running the *same* gains at 200 Hz incurs two rate penalties: the extra ≈2 ms of hold delay costs ≈28° of phase at the crossover, and the filtered derivative—whose time constant $T_s\alpha/(1-\alpha)$ scales with the loop period T_s—costs a further ≈24°, together dropping the phase margin to ≈34° (gain margin ≈2.4 dB): still stable, but with the comfortable headroom gone. Restoring the multi-rate margin at 200 Hz then forces the coupling gain down to about 0.65× (K_p^s : 3.6 → 2.4), cutting the difference-loop crossover from ≈39 to ≈21 Hz—the concrete meaning of a single-rate loop that must trade coupling gain, and with it disturbance-rejection bandwidth, for stability-margin headroom; the empirical single-rate retune bears this out (Ours 7.66 rpm RMSE vs the multi-rate 6.26 rpm, Table 8). This margin illustration holds only where the tuner chose a firm coupling gain: K_p^s spans ≈0.6–3.6 across the three seeds, and at the two lower-gain seeds the difference loop crosses over below 8 Hz—so gentle that the loop rate barely moves its margin at all, consistent with the flat objective. Stability and performance across the family therefore rest on the nonlinear evaluation of Section 6.4 as much as on this linear margin, which quantifies the mechanism at a representative firm-gain seed rather than bounding the family. The margins here are computed from the identified first-order difference-mode model (13) driven by the deployed gains, not read off a measured closed-loop frequency response; a swept-sine measurement of the difference channel on the vehicle would confirm them directly.

Table 8: Single-Rate vs. Multi-Rate, Each Coupling Law Re-Tuned at Its Loop Rate (mean ± SD over three seeds; single-rate 200 Hz, multi-rate 1 kHz; better of each pair in bold)

Law	Rate	Clean RMSE (rpm)	Clean peak (rpm)
CCC	200 Hz	11.44 ± 1.76	93.8 ± 23.7
CCC	1 kHz	**6.58** ± 2.03	**35.2** ± 7.4
SMC	200 Hz	6.61 ± 0.54	**69.3** ± 6.5
SMC	1 kHz	**6.05** ± 1.50	71.4 ± 22.2
Ours	200 Hz	7.66 ± 1.97	65.2 ± 19.4
Ours	1 kHz	**6.26** ± 1.03	**51.5** ± 17.6

The fast loop helps CCC decisively (2.6σ RMSE, 3.3σ peak) and our controller within spread (0.9σ, 0.7σ); the boundary-layer super-twisting law is nearly flat on RMSE (6.61 → 6.05) and no better on peak, so it does not benefit from the faster rate. Each row is an independent GA re-tune at its own loop rate (all twelve genes free), so the single-rate CCC and Ours rows differ in more than the loop rate—the wide seed-to-seed SD is also why single-rate CCC (11.44 ± 1.76) overlaps no-sync (10.67 ± 1.24); the super-twisting boundary layer Φ=4 rpm is held fixed across both rates.

6.6. Comparison with Reported Synchronization Results

Table 9 places the result among recent dual- and multi-motor synchronization studies. The comparison is *suggestive only* and not like-for-like: each work uses a different plant, rating, and disturbance, so the numbers are not directly comparable and we claim no cross-plant superiority—the controlled result is Table 7. For comparability with the cited *peak* errors, our held-out peak inter-motor error (≈51.5 rpm) sits on the higher end of the reported range (≈7–95 rpm)—reflecting a continuously terrain-perturbed running condition rather than the smoother single-load benches of the cited works—while our clean-tracking RMSE (≈6.3 rpm at a 3000 rpm ceiling) is comparatively low.

6.7. Limitations

Several limitations bound these results and frame the next steps. *(i) Estimator characterized at one operating point.* The per-motor INL calibration and the ≈2.5 rpm noise floor it achieves are measured (Section 3.3), but at a single calibration speed and temperature: the lookup table captures each encoder as it was at commissioning, and temperature-induced INL drift is not tracked. More consequentially for the claim this paper rests on, the observer bandwidth is never *varied*—every arm, seed and table row here uses the same ≈80 Hz estimator—so the loop-rate results below establish the benefit of a faster loop *at that bandwidth* rather than tracing out the bound the bandwidth imposes; a bandwidth sweep against a rate sweep is the experiment

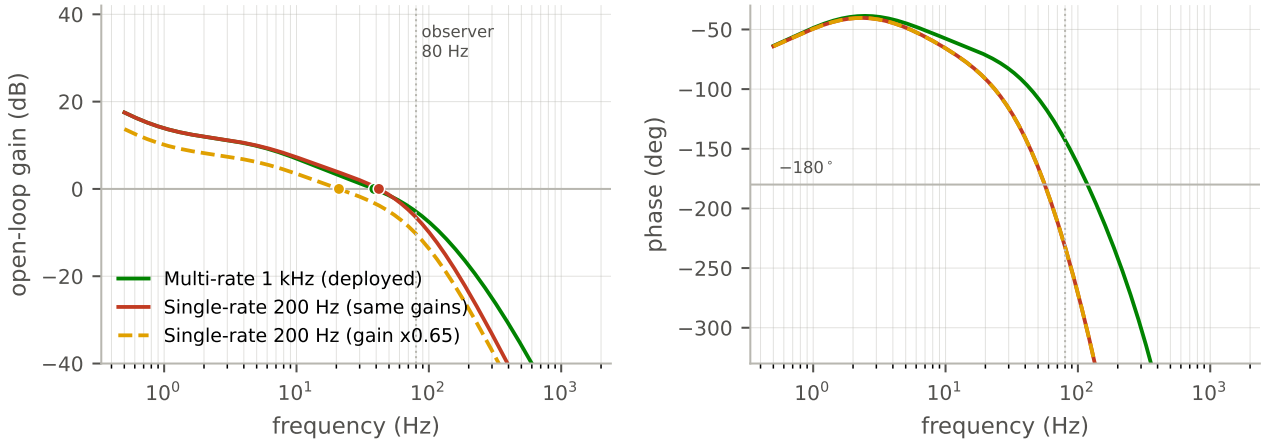


Figure 9: Difference-channel loop transfer (13) (gain, left; phase, right) at the representative firm-gain seed. The 1 kHz loop (green) crosses 0 dB at ≈39 Hz with ≈86° phase margin; the *same* gains at 200 Hz (red) retain ≈34°—stable, but the headroom is gone. Matching the multi-rate margin at 200 Hz (dashed) needs about 0.65× the coupling gain and gives ≈21 Hz crossover. Dotted line: the 80 Hz observer corner. Markers: gain crossovers.

Table 9
Reported Inter-Motor Synchronization Error, Selected Studies[a]

Study	Plat.	Motors (rated rpm)	Method	Test condition	Sync. err. (rpm)
Chen (Chen et al., 2018)	HW	2 PMSM (1500)	Improved CCC	10 N m sudden load	95
Zhu (Zhu et al., 2020)	HW	2 PMSM	2-GFTSMC	40 N m load step	23
Wu (Wu et al., 2025)	Sim	2 PMSM	GFTSMC+DOB	friction disturbance	11
Zou (Zou and Zhao, 2020)	Sim	2 steering BLDC	CC-SMC	0.01 N m load	23.7
Wang (Wang et al., 2023)	Sim	4 PMSM (1000)	RCC-ABC	clean (no disturbance)	7.2[c]
This work	HW[b]	2 BLDC (3000)	Multi-rate CCC + per-motor FF	terrain + one-sided load	51.5 pk; 6.3 RMSE

[a]Different plants, ratings, and disturbances; suggestive context only—the controlled comparison is Table 7. Each cited value is the source's reported peak inter-motor speed error under its stated condition. [b]We list our held-out *peak* inter-motor error (≈51.5 rpm) for like-metric comparison with the cited peaks, and the clean-tracking RMSE (≈6.3 rpm) alongside it, both over three GA seeds; "(3000)" is the maximum commanded speed, not a rated speed. [c](Wang et al., 2023) reports a synchronous-error standard deviation rather than a peak.

Abbreviations: HW=hardware, Sim=simulation; CCC=cross-coupled control; (2-)GFTSMC=(double) global fast-terminal SMC; DOB=disturbance observer; CC-SMC=cross-coupled SMC; RCC-ABC=relative cross-coupling tuned by artificial bee colony.

that would. *(ii) Heading is a lower bound.* Heading is a no-slip kinematic estimate from the motor-shaft speeds, with no inertial sensor on the vehicle; crucially, a drive-sprocket radius mismatch from wear is invisible to a motor-shaft encoder, so real path error can exceed (11). *(iii) Scope of the contribution.* The performance lever is the multi-rate loop, not the per-motor feedforward: at a matched rate our controller and conventional cross-coupling are indistinguishable (Section 6.5), so the feedforward is a transparent, low-overhead component rather than a source of improvement, and the controller is essentially conventional cross-coupling made multi-rate. *(iv) Loosely-determined gains, single tuner, single test course.* The objective is nearly flat in gain space, so the deployed feedback gains are one of many near-equivalent sets—the synchronization K_p^s alone spans ≈0.6–3.6 across the three seeds—and tuning is a single three-seed campaign rather than continuous adaptation. Every run reported here was taken on one test course under one set of ambient and battery conditions, so the reported spread is a repeatability figure on that course and does not bound terrain-to-terrain variation; a robustness sweep over distinct surfaces and a full multi-terrain *re-tuning* campaign on the reduced-plan gains are future work. The same flatness reaches the results: the uncoupled one-sided-load response varies widely between tunings (24.9 ± 15.8 rpm, Table 6), so the reported advantage over the independent baseline describes the tuner's typical outcome rather than a guaranteed one. Under the stricter paired-by-seed criterion (Section 6.4) the controller-to-controller margins are trends rather than resolved separations, so the comparison establishes a ranking, not statistical dominance over the coupled baselines. The super-twisting and cross-coupling baselines are our own same-plant re-implementations, not transcriptions of the source studies' hardware. *(v) Straight-line scope.* The synchronization law (8) regulates the inter-motor speed *difference* to zero, the straight-line objective (equal references) to which this evaluation is confined; a coordinated turn—a commanded nonzero speed difference—would regulate the difference about its reference value (Section 4) and is left to future work.

7. Conclusion and Future Work

We presented a GA-tuned, multi-rate cross-coupled speed-synchronization controller for a tracked skid-steer vehicle driven by two BLDC motors, realized as a distributed one-microcontroller-per-motor architecture with a dedicated 1 ms inter-MCU speed exchange and a 1 kHz multi-rate synchronization loop closed on a capacitive-encoder tracking-observer speed estimate. Every feedback and per-motor feedforward gain is tuned *online* on the assembled vehicle by a genetic algorithm, because the inter-drive mechanical asymmetries (track tension, friction, wear) cannot be modeled beforehand. Evaluated over three tuning seeds on held-out operating points and a one-sided load the tuner never saw, the controller reduces held-out clean-tracking RMSE by 41% and one-sided-load sustained error by 65% versus an independent-drive baseline (the clean-tracking gain exceeding twice the pooled run-to-run SD on a single test course, and the only one to survive a paired-by-seed test), and reduces clean-tracking RMSE and peak by 45% and 45% *in the mean* versus conventional single-rate cross-coupling—entirely a loop-rate effect (the two are indistinguishable at a matched rate) whose RMSE and peak margins do not separate under a paired-by-seed test. A same-plant comparison identifies the multi-rate loop as the performance lever and shows that it pays off only once the low-latency observer lifts the speed-estimate bandwidth—the useful loop rate is bounded by the estimator, which is set by the sensor. The transparent per-motor-feedforward PID law is competitive with a super-twisting sliding-mode baseline—leading on clean tracking and peak and matching it within run-to-run spread on the held-out load—without a runtime plant model or sliding surface.

Future work is led by continuous *adaptive* re-tuning as the mechanical asymmetries drift with wear, together with a bench-validated per-motor encoder non-linearity calibration, more aggressive maneuvers and varied terrain, and extension beyond two drives.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of generative AI use

During the preparation of this work the authors used an AI coding and writing assistant to help develop the firmware , and draft and edit the manuscript. The authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

Data availability

The data supporting the findings of this study will be made available by the corresponding author on reasonable request.

CRediT authorship contribution statement

Md Monirul Islam: Conceptualization, Methodology, Software, Investigation, Writing – original draft. **Mainul Islam:** Software, Validation, Data curation, Writing – review & editing. **Md Shakil Tanvir:** Software, Supervision, Writing – review & editing.

References

Abed, M.A.N., Hashim, N.M., Nasar, Y., Hanfesh, A.O., Altahir, A.A.R., 2025. Genetic Algorithm-Tuned PID for Sensorless PMSM Speed Control Using a Second-Order Sliding Mode Observer. Journal of Robotics and Control (JRC) 6. URL: https://journal.umy.ac.id/index.php/jrc/article/view/27013.

Amornwongpeeti, S., Ekpanyapong, M., Chayopitak, N., Monteiro, J.L., Martins, J.S., Afonso, J.L., 2015. A Single Chip FPGA-Based Cross-Coupling Multi-Motor Drive System. IEICE Electronics Express 12. doi:10.1587/elex.12.20150383.

Araki, M., Yamamoto, K., 1986. Multivariable Multirate Sampled-Data Systems: State-Space Description, Transfer Characteristics, and Nyquist Criterion. IEEE Transactions on Automatic Control 31, 145–154. doi:10.1109/TAC.1986.1104205.

Ashraf, A., Ahsan, H., Arshad, M., 2021. Motor Speed Synchronization of Mobile Robot Using PI Controller, in: 2021 Int. Conf. on Digital Futures and Transformative Technologies (ICoDT2). doi:10.1109/ICoDT252288.2021.9441518.

Chen, W., Liang, J., Shi, T., 2018. Speed Synchronous Control of Multiple Permanent Magnet Synchronous Motors Based on an Improved Cross-Coupling Structure. Energies (MDPI) doi:10.3390/en11020282.

Choi, Y.B., Lee, S.H., Jung, D.H., 2024. Controller Design Based on a Synchronous Error Compensator for Wheeled Cross-coupled Systems. International Journal of Control, Automation and Systems doi:10.1007/s12555-023-0687-x.

Dai, Y., Zhang, L., Xu, D., Chen, Q., Yan, X.G., 2022. Anti-Disturbance Cooperative Fuzzy Tracking Control of Multi-PMSMs Low-Speed Urban Rail Traction Systems. IEEE Transactions on Transportation Electrification doi:10.1109/TTE.2021.3133316.

Dakheel, H.S., Abdullah, Z.B., Shneen, S.W., 2023. Advanced optimal GA-PID controller for BLDC motor. Bulletin of Electrical Engineering and Informatics 12, 2077–2086. doi:10.11591/eei.v12i4.4649.

Deraz, S.A., 2014. Genetic Tuned PID Controller Based Speed Control of DC Motor Drive. International Journal of Engineering Trends and Technology (IJETT) 17. doi:10.14445/22315381/IJETT-V17P219.

Feng, L., Koren, Y., Borenstein, J., 1993. Cross-coupling motion controller for mobile robots. IEEE Control Systems Magazine doi:10.1109/37.248002.

Fujimoto, H., Hori, Y., 2002. High-Performance Servo Systems Based on Multirate Sampling Control. Control Engineering Practice 10, 773–781. doi:10.1016/S0967-0661(02)00010-2.

Fujimoto, H., Hori, Y., Kawamura, A., 2001. Perfect Tracking Control Based on Multirate Feedforward Control with Generalized Sampling Periods. IEEE Transactions on Industrial Electronics 48, 636–644. doi:10.1109/41.925591.

Garimella, R.C., et al., 2012. Modeling and PID control of the brushless DC motor with the help of Genetic Algorithm, in: 2012 IEEE Int. Conf. on Advances in Engineering, Science and Management (ICAESM). doi:10.1109/ICAESM.2012.6216186.

Ha, V., Thuong, T., Truc, L., 2024. Trajectory tracking control based on genetic algorithm and proportional integral derivative controller for two-wheel mobile robot. Bulletin of Electrical Engineering and Informatics doi:10.11591/eei.v13i4.7847.

Hassanat, A., Almohammadi, K., Alkafaween, E., Abunawas, E., Hammouri, A., Prasath, V.B.S., 2019. Choosing Mutation and Crossover Ratios for Genetic Algorithms — A Review with a New Dynamic Approach. Information (MDPI) doi:10.3390/info10120390.

Huang, H., Tu, Q., Jiang, C., Li, L., Li, P., Zhang, H., 2018. Dual motor drive vehicle speed synchronization and coordination control strategy,

in: AIP Conference Proceedings, p. 040005. doi:10.1063/1.5033669.

Huang, H., Tu, Q., Jiang, C., Pan, M., Zhu, C., 2021. An Electronic Line-Shafting Control Strategy Based on Sliding Mode Observer for Distributed Driving Electric Vehicles. IEEE Access doi:10.1109/ACCESS.2021.3062829.

Ibrahim, M.A., Mahmood, A.K., Sultan, N.S., 2019. Optimal PID controller of a brushless DC motor using genetic algorithm. International Journal of Power Electronics and Drive Systems (IJPEDS) 10, 822–830. doi:10.11591/ijpeds.v10.i2.pp822-830.

Jerković Štil, V., Varga, T., Benšić, T., Barukčić, M., 2020. A Survey of Fuzzy Algorithms Used in Multi-Motor Systems Control. Electronics (MDPI) 9, 1788. doi:10.3390/electronics9111788.

Joseph, S.B., Dada, E.G., Abidemi, A., Oyewola, D.O., Khammas, B.M., 2022. Metaheuristic algorithms for PID controller parameters tuning: review, approaches and open problems. Heliyon doi:10.1016/j.heliyon.2022.e09399.

Jung, D., Kim, S., 2023. A Passive Decomposition Based Robust Synchronous Motion Control of Multi-Motors and Experimental Verification. Sensors (MDPI) doi:10.3390/s23177603.

Karnavas, Y.L., Topalidis, A.S., Drakaki, M., 2019. Development and Implementation of a Low Cost µC-Based Brushless DC Motor Sensorless Controller: A Practical Analysis of Hardware and Software Aspects. Electronics (MDPI) doi:10.3390/electronics8121456.

Koren, Y., 1980. Cross-Coupled Biaxial Computer Control for Manufacturing Systems. Journal of Dynamic Systems, Measurement, and Control (ASME) doi:10.1115/1.3149612.

Kuang, Z., Gao, H., Tomizuka, M., 2020. Precise Linear-Motor Synchronization Control Via Cross-Coupled Second-Order Discrete-Time Fractional-Order Sliding Mode. IEEE/ASME Transactions on Mechatronics doi:10.1109/TMECH.2020.3019883.

Lan, C., Wang, H., Deng, X., Zhang, X.F., Song, H., 2023. Multi-motor position synchronization control method based on non-singular fast terminal sliding mode control. PLOS ONE doi:10.1371/journal.pone.0281721.

Le, V., Tran, D., Nhat, T., 2022. Synchronous Control of Dual Motor with Master-Slave and Cross-Coupling Methods, in: Lecture Notes in Networks and Systems (Computational Intelligence Methods for Green Technology and Sustainable Development), Springer. doi:10.1007/978-3-031-19694-2_52.

Levant, A., 1993. Sliding order and sliding accuracy in sliding mode control. International Journal of Control 58, 1247–1263. doi:10.1080/00207179308923053.

Li, B., Chai, H., Hou, L., 2025. Fixed-time leader-following speed consensus control for multi-PMSMs based on multi-agent systems consensus. Scientific Reports doi:10.1038/s41598-025-18061-3.

Li, L.B., Sun, L.L., Zhang, S.Z., Yang, Q.Q., 2015. Speed tracking and synchronization of multiple motors using ring coupling control and adaptive sliding mode control. ISA Transactions doi:10.1016/j.isatra.2015.07.010.

Lin, S., Cai, Y., Yang, B., Zhang, W., 2017. Electrical line-shafting control for motor speed synchronisation using sliding mode controller and disturbance observer. IET Control Theory & Applications doi:10.1049/iet-cta.2016.0924.

Liu, Z., Lin, W., Yu, X., Rodríguez-Andina, J.J., Gao, H., 2022. Approximation-Free Robust Synchronization Control for Dual-Linear-Motors-Driven Systems With Uncertainties and Disturbances. IEEE Transactions on Industrial Electronics doi:10.1109/TIE.2021.3137619.

Mahmood, M.H., Ali, O.F., Mnati, M.J., Sabbar, B.M., Van Den Bossche, A., 2025. Intelligent PID Parameter Tuning for BLDC Motors by Using Genetic Algorithms. Journal Européen des Systèmes Automatisés (JESA) 58. doi:10.18280/jesa.580714.

Megalingam, R., Manoharan, S., Mohandas, S., Kamalasanan, A., 2025. Design and Development of a Controller PCB for a Dual BLDC-Based Differential Drive Rover, in: Lecture Notes in Networks and Systems, Springer. doi:10.1007/978-981-96-0228-5_10.

Miao, Z., He, Y., Li, H., Wang, Y., 2023. Dual-PMSM speed synchronization control based on unified prediction model. Journal of Measurement Science and Instrumentation doi:10.62756/jmsi.1674-8042.2023036.

Mu, Y., Qi, L., Sun, M., Han, W., 2024. An Improved Deviation Coupling Control Method for Speed Synchronization of Multi-Motor Systems. Applied Sciences (MDPI) doi:10.3390/app14125300.

Niu, F., Sun, K., Huang, S., Hu, Y., Liang, D., Fang, Y., 2023. A Review on Multimotor Synchronous Control Methods. IEEE Transactions on Transportation Electrification 9. doi:10.1109/TTE.2022.3168647.

Reda, M., Onsy, A., Haikal, A., Ghanbari, A., 2025. Motor Speed Control of Four-wheel Differential Drive Robots Using a New Hybrid Moth-flame Particle Swarm Optimization (MFPSO) Algorithm. Journal of Intelligent & Robotic Systems doi:10.1007/s10846-025-02228-1.

Romero Segovia, V., Hägglund, T., Åström, K., 2014. Measurement Noise Filtering for Common PID Tuning Rules. Control Engineering Practice 32, 43–63. doi:10.1016/j.conengprac.2014.07.005.

Shafique, F., Fakhar, M.S., Rasool, A., Kashif, S.A.R., Suhail, M., 2024. Analyzing the performance of metaheuristic algorithms in speed control of brushless DC motor: Implementation and statistical comparison. PLOS ONE doi:10.1371/journal.pone.0310080.

Shi, T., Liu, H., Geng, Q., Xia, C., 2016. Improved relative coupling control structure for multi-motor speed synchronous driving system. IET Electric Power Applications doi:10.1049/iet-epa.2015.0515.

Tanvir, M.S., Ahmed, F., Islam, M.M., Islam, M., Hasan, M., Rashid, R.U., 2024. Optimizing Master-Slave Broadcast Communication in Multi-Node Network Using RS485 Standard, in: 2024 27th International Conference on Computer and Information Technology (ICCIT), IEEE, Cox's Bazar, Bangladesh. URL: https://ieeexplore.ieee.org/document/11022430.

Valenzuela, M., Lorenz, R., 2001. Electronic line-shafting control for paper machine drives. IEEE Transactions on Industry Applications doi:10.1109/28.903141.

Wang, H., Du, H., Cui, Q., Liu, P., Ma, X., 2023. A multi-motor speed synchronization control enhanced by artificial bee colony algorithm. Measurement and Control doi:10.1177/00202940221122160.

Wu, K., Zhang, Y., Qi, Y., Shi, W., 2025. Control Strategy for Improving Dual Motor Synchronization Accuracy: Cross Coupling Method Based on Improved Global Fast Terminal Sliding Mode Control and Disturbance Observer. Applied Sciences (MDPI) doi:10.3390/app15041915.

Zhang, C., Niu, M., He, J., Zhao, K., Wu, H., Zhang, M., 2016. Robust synchronous control of multi-motor integrated with artificial potential field and sliding mode variable structure. IEEE Access doi:10.1109/ACCESS.2016.2636224.

Zhang, K., Wang, L., Fang, X., 2022. Robust decoupled synchronization control of dual-motor servo systems by mechanical parameter identification. IET Power Electronics doi:10.1049/pel2.12320.

Zhang, X., Hu, H., Wang, H., Wang, Z., 2024. Overview of position synchronous control technology for multi-motor system. Systems Science & Control Engineering doi:10.1080/21642583.2024.2427074.

Zhao, C., Hua, Z., 2022. Design of Motor Speed Control System Based on STM32 Microcontroller, in: 2022 IEEE Int. Conf. on Computation, Big-Data and Engineering (ICCBE). doi:10.1109/ICCBE56101.2022.9888225.

Zhu, C., Tu, Q., Jiang, C., Pan, M., Huang, H., 2020. A Cross Coupling Control Strategy for Dual-Motor Speed Synchronous System Based on Second Order Global Fast Terminal Sliding Mode Control. IEEE Access doi:10.1109/ACCESS.2020.3042256.

Zou, S., Zhao, W., 2020. Synchronization and stability control of dual-motor intelligent steer-by-wire vehicle. Mechanical Systems and Signal Processing doi:10.1016/j.ymssp.2020.106925.